University of Asia Pacific

Department of Computer Science and Engineering

Course Code : CSE 450 (VLSI Lab)

Final Project | Spring 2026

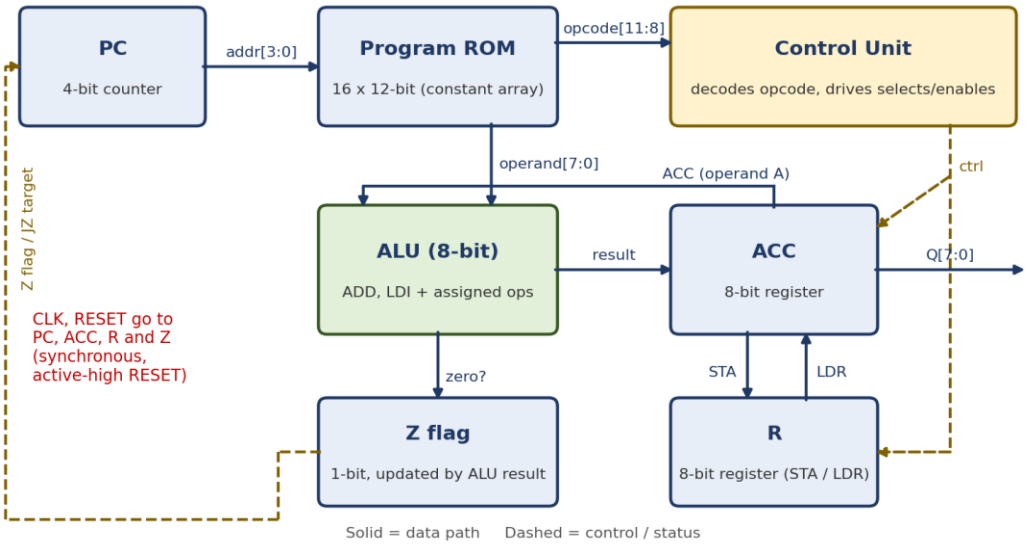
Total Marks	50 (Viva → 30, Report & VrC → 10, Presentation → 10)		
Student Name		**ID**	

Final Project: Mini 8-bit Accumulator CPU

Objective. Design and implement a small single-cycle 8-bit accumulator CPU in VHDL. The CPU supports only five instructions: two are mandatory for every student, and the remaining three are assigned to you according to your Student ID (see the “Instruction Set Assignment” page: the last digit of your Student ID decides your set). You must also verify the design with a testbench that runs the program given for your set in Appendix B.

1. System Architecture

The CPU is built from the blocks shown below. Every sequential block (PC, ACC, R, Z) is clocked on the rising edge of CLK and cleared by a synchronous, active-high RESET.



Block	Description
PC	4-bit program counter. Holds the address of the instruction being executed. After every instruction it becomes PC + 1, except when JZ is taken.
Program ROM	16 locations x 12 bits, written as a constant array in VHDL. It is read combinationally using PC (no clock needed). Unused locations contain 0x000 (NOP).
Control Unit	Decodes opcode[11:8] and generates the ALU function select and the load/enable signals for ACC, R, Z and PC.
ALU	8-bit combinational unit working on ACC and the 8-bit operand. Only the operations of your own instruction set are required.
ACC, R, Z	ACC: 8-bit accumulator, also the CPU output Q[7:0]. R: 8-bit holding register (only if your set has STA / LDR). Z: 1-bit flag, 1 when the value written to ACC is 0x00.

2. Instruction Format

Each instruction is 12 bits: **instr[11:8] = opcode** (4 bits) and **instr[7:0] = operand** (8-bit immediate). Every instruction takes exactly one clock cycle (fetch, decode and execute happen in the same cycle, the result is written at the rising edge).

Mandatory instructions (all students):

Mnemonic	Opcode	Operation	Flags
LDI imm	0001	ACC ← imm	Z updated
ADD imm	0010	ACC ← ACC + imm (8-bit, carry discarded)	Z updated

Instruction pool (you implement only the three assigned to you):

Mnemonic	Opcode	Operation	Flags
SUB imm	0011	ACC ← ACC − imm (modulo 256)	Z updated
AND imm	0100	ACC ← ACC AND imm	Z updated
OR imm	0101	ACC ← ACC OR imm	Z updated
XOR imm	0110	ACC ← ACC XOR imm	Z updated
NOT	0111	ACC ← NOT ACC (operand ignored)	Z updated
SHL	1000	ACC ← ACC shifted left by 1, 0 enters LSB (operand ignored)	Z updated
SHR	1001	ACC ← ACC shifted right by 1, 0 enters MSB (operand ignored)	Z updated
STA	1010	R ← ACC (operand ignored, ACC unchanged)	Z unchanged
LDR	1011	ACC ← R (operand ignored)	Z updated
JZ addr	1100	If Z = 1 then PC ← operand[3:0], else PC ← PC + 1	Z unchanged

Note: if your set contains LDR it also contains STA. Any opcode not in your set (including 0000) behaves as NOP: PC ← PC + 1 and nothing else changes.

3. Reset Behaviour

RESET = 1 at a rising edge: PC ← 0, ACC ← 0x00, R ← 0x00, Z ← 1 (RESET has priority over everything). **RESET = 0 at a rising edge:** execute the instruction at ROM[PC]. Between clock edges all registers keep their values.

4. Implementation Requirements

- Use VHDL with IEEE.STD_LOGIC_1164.ALL (IEEE.NUMERIC_STD is allowed for arithmetic).
- Modular design: separate entities for PC, Program ROM, Control Unit, ALU and ACC/R/Z, connected in a top-level CPU entity.
- RESET must be synchronous and active-high. All state changes must occur on the rising edge of CLK.
- The top level must expose CLK, RESET, Q[7:0] (the ACC), the Z flag and, for checking, PC[3:0].
- Implement only the five instructions of your own set. The ROM must hold exactly the Appendix B program of your set; the testbench must generate a clock, apply RESET for one rising edge, and run until the last instruction has executed.

Instruction Set Assignment

How to find your set: take the **last digit of your Student ID** (ID mod 10). That digit is your set number in the table below. Example: ID 22201014 → last digit 4 → Set 4.

Every student implements the two mandatory instructions **LDI** and **ADD** plus the three assigned instructions of his/her set. Total = 5 instructions.

Last digit of ID	Set	Mandatory	Assigned	Complete instruction set of the CPU
0	**Set 0**	LDI, ADD	AND, JZ, NOT	LDI, ADD, AND, JZ, NOT
1	**Set 1**	LDI, ADD	SUB, AND, OR	LDI, ADD, SUB, AND, OR
2	**Set 2**	LDI, ADD	XOR, NOT, SHL	LDI, ADD, XOR, NOT, SHL
3	**Set 3**	LDI, ADD	SHR, SUB, XOR	LDI, ADD, SHR, SUB, XOR
4	**Set 4**	LDI, ADD	STA, LDR, AND	LDI, ADD, STA, LDR, AND

Last digit of ID	Set	Mandatory	Assigned	Complete instruction set of the CPU
5	**Set 5**	LDI, ADD	OR, NOT, SHR	LDI, ADD, OR, NOT, SHR
6	**Set 6**	LDI, ADD	SUB, JZ, SHL	LDI, ADD, SUB, JZ, SHL
7	**Set 7**	LDI, ADD	AND, XOR, SHR	LDI, ADD, AND, XOR, SHR
8	**Set 8**	LDI, ADD	STA, LDR, SUB	LDI, ADD, STA, LDR, SUB
9	**Set 9**	LDI, ADD	NOT, SHL, JZ	LDI, ADD, NOT, SHL, JZ

Appendix B: Required Test Program and Expected Results

Store the program of your set (Appendix B tables are ordered by set number 0–9) in the Program ROM. Apply RESET for one rising edge, then run the CPU for one clock cycle per executed instruction. The ACC, Z, R and next-PC columns show the values after the corresponding rising edge. Addresses that are not listed contain 0x000 (NOP). Operands are in hexadecimal. For NOT, SHL, SHR, STA and LDR the operand field is 0x00.

ROM encoding: instr[11:0] = opcode (4 bit) & operand (8 bit). Example: LDI 0x3C = 0001 00111100 = 0x13C.

Set 0: LDI, ADD, AND, JZ, NOT

ROM contents: 0: LDI 0xA5 → 0x1A5 | 1: AND 0x5A → 0x45A | 2: JZ 0x05 → 0xC05 | 3: ADD 0x01 → 0x201 | 4: ADD 0x01 → 0x201 | 5: NOT → 0x700 | 6: AND 0x0F → 0x40F | 7: JZ 0x00 → 0xC00 | 8: ADD 0x01 → 0x201

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0xA5	0xA5	0	10100101	1
2	1	AND 0x5A	0x00	1	00000000	2
3	2	JZ 0x05 (taken)	0x00	1	00000000	5
4	5	NOT	0xFF	0	11111111	6
5	6	AND 0x0F	0x0F	0	00001111	7
6	7	JZ 0x00 (not taken)	0x0F	0	00001111	8
7	8	ADD 0x01	0x10	0	00010000	9

Set 1: LDI, ADD, SUB, AND, OR

ROM contents: 0: LDI 0x3C → 0x13C | 1: ADD 0x0F → 0x20F | 2: SUB 0x0B → 0x30B | 3: AND 0xF0 → 0x4F0 | 4: OR 0x0A → 0x50A | 5: SUB 0x4A → 0x34A | 6: SUB 0x01 → 0x301

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x3C	0x3C	0	00111100	1
2	1	ADD 0x0F	0x4B	0	01001011	2
3	2	SUB 0x0B	0x40	0	01000000	3
4	3	AND 0xF0	0x40	0	01000000	4
5	4	OR 0x0A	0x4A	0	01001010	5
6	5	SUB 0x4A	0x00	1	00000000	6
7	6	SUB 0x01	0xFF	0	11111111	7

Set 2: LDI, ADD, XOR, NOT, SHL

ROM contents: 0: LDI 0x55 → 0x155 | 1: XOR 0xFF → 0x6FF | 2: NOT → 0x700 | 3: SHL → 0x800 | 4: ADD 0x56 → 0x256 | 5: XOR 0x81 → 0x681 | 6: SHL → 0x800

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x55	0x55	0	01010101	1
2	1	XOR 0xFF	0xAA	0	10101010	2
3	2	NOT	0x55	0	01010101	3
4	3	SHL	0xAA	0	10101010	4
5	4	ADD 0x56	0x00	1	00000000	5
6	5	XOR 0x81	0x81	0	10000001	6
7	6	SHL	0x02	0	00000010	7

Set 3: LDI, ADD, SHR, SUB, XOR

ROM contents: 0: LDI 0x80 → 0x180 | 1: SHR → 0x900 | 2: SUB 0x10 → 0x310 | 3: ADD 0x0F → 0x20F | 4: XOR 0x3F → 0x63F | 5: ADD 0x07 → 0x207 | 6: SHR → 0x900

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x80	0x80	0	10000000	1
2	1	SHR	0x40	0	01000000	2
3	2	SUB 0x10	0x30	0	00110000	3
4	3	ADD 0x0F	0x3F	0	00111111	4
5	4	XOR 0x3F	0x00	1	00000000	5
6	5	ADD 0x07	0x07	0	00000111	6
7	6	SHR	0x03	0	00000011	7

Set 4: LDI, ADD, STA, LDR, AND

ROM contents: 0: LDI 0x1E → 0x11E | 1: STA → 0xA00 | 2: ADD 0x22 → 0x222 | 3: AND 0x60 → 0x460 | 4: LDR → 0xB00 | 5: ADD 0x01 → 0x201 | 6: AND 0x0F → 0x40F | 7: STA → 0xA00 | 8: LDI 0x00 → 0x100 | 9: LDR → 0xB00

Cycle	PC	Instruction executed	ACC (hex)	Z	R (hex)	ACC (bin)	Next PC
0	-	RESET = 1	0x00	1	0x00	00000000	0
1	0	LDI 0x1E	0x1E	0	0x00	00011110	1
2	1	STA	0x1E	0	0x1E	00011110	2
3	2	ADD 0x22	0x40	0	0x1E	01000000	3
4	3	AND 0x60	0x40	0	0x1E	01000000	4
5	4	LDR	0x1E	0	0x1E	00011110	5
6	5	ADD 0x01	0x1F	0	0x1E	00011111	6
7	6	AND 0x0F	0x0F	0	0x1E	00001111	7
8	7	STA	0x0F	0	0x0F	00001111	8
9	8	LDI 0x00	0x00	1	0x0F	00000000	9
10	9	LDR	0x0F	0	0x0F	00001111	10

Set 5: LDI, ADD, OR, NOT, SHR

ROM contents: 0: LDI 0x0F → 0x10F | 1: OR 0xA0 → 0x5A0 | 2: NOT → 0x700 | 3: SHR → 0x900 | 4: ADD 0xD8 → 0x2D8 | 5: OR 0x81 → 0x581 | 6: SHR → 0x900

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x0F	0x0F	0	00001111	1
2	1	OR 0xA0	0xAF	0	10101111	2
3	2	NOT	0x50	0	01010000	3
4	3	SHR	0x28	0	00101000	4
5	4	ADD 0xD8	0x00	1	00000000	5
6	5	OR 0x81	0x81	0	10000001	6
7	6	SHR	0x40	0	01000000	7

Set 6: LDI, ADD, SUB, JZ, SHL

ROM contents: 0: LDI 0x02 → 0x102 | 1: SUB 0x01 → 0x301 | 2: JZ 0x07 → 0xC07 | 3: SUB 0x01 → 0x301 | 4: JZ 0x07 → 0xC07 | 5: ADD 0xAA → 0x2AA | 6: ADD 0xAA → 0x2AA | 7: LDI 0x21 → 0x121 | 8: SHL → 0x800 | 9: ADD 0x01 → 0x201

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x02	0x02	0	00000010	1
2	1	SUB 0x01	0x01	0	00000001	2
3	2	JZ 0x07 (not taken)	0x01	0	00000001	3
4	3	SUB 0x01	0x00	1	00000000	4
5	4	JZ 0x07 (taken)	0x00	1	00000000	7
6	7	LDI 0x21	0x21	0	00100001	8
7	8	SHL	0x42	0	01000010	9
8	9	ADD 0x01	0x43	0	01000011	10

Set 7: LDI, ADD, AND, XOR, SHR

ROM contents: 0: LDI 0xF3 → 0x1F3 | 1: AND 0x3C → 0x43C | 2: XOR 0x11 → 0x611 | 3: SHR → 0x900 | 4: ADD 0x30 → 0x230 | 5: XOR 0x40 → 0x640

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0xF3	0xF3	0	11110011	1
2	1	AND 0x3C	0x30	0	00110000	2
3	2	XOR 0x11	0x21	0	00100001	3
4	3	SHR	0x10	0	00010000	4
5	4	ADD 0x30	0x40	0	01000000	5
6	5	XOR 0x40	0x00	1	00000000	6

Set 8: LDI, ADD, STA, LDR, SUB

ROM contents: 0: LDI 0x64 → 0x164 | 1: STA → 0xA00 | 2: SUB 0x14 → 0x314 | 3: ADD 0x0A → 0x20A | 4: STA → 0xA00 | 5: SUB 0x5A → 0x35A | 6: LDR → 0xB00 | 7: SUB 0x60 → 0x360

Cycle	PC	Instruction executed	ACC (hex)	Z	R (hex)	ACC (bin)	Next PC
0	-	RESET = 1	0x00	1	0x00	00000000	0
1	0	LDI 0x64	0x64	0	0x00	01100100	1
2	1	STA	0x64	0	0x64	01100100	2
3	2	SUB 0x14	0x50	0	0x64	01010000	3
4	3	ADD 0x0A	0x5A	0	0x64	01011010	4
5	4	STA	0x5A	0	0x5A	01011010	5
6	5	SUB 0x5A	0x00	1	0x5A	00000000	6
7	6	LDR	0x5A	0	0x5A	01011010	7
8	7	SUB 0x60	0xFA	0	0x5A	11111010	8

Set 9: LDI, ADD, NOT, SHL, JZ

ROM contents: 0: LDI 0xFF → 0x1FF | 1: NOT → 0x700 | 2: JZ 0x05 → 0xC05 | 3: ADD 0x77 → 0x277 | 4: ADD 0x77 → 0x277 | 5: ADD 0x11 → 0x211 | 6: SHL → 0x800 | 7: JZ 0x00 → 0xC00 | 8: SHL → 0x800

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0xFF	0xFF	0	11111111	1
2	1	NOT	0x00	1	00000000	2
3	2	JZ 0x05 (taken)	0x00	1	00000000	5
4	5	ADD 0x11	0x11	0	00010001	6
5	6	SHL	0x22	0	00100010	7
6	7	JZ 0x00 (not taken)	0x22	0	00100010	8
7	8	SHL	0x44	0	01000100	9

Submission Guidelines

Submit your VHDL source files, testbench and a project report within the 2-week period. Include this project description (question, assignment page and Appendix B) at the front of your report and start your own report on a new page after it. The report must include, but is not limited to, the following points.

- Your Student ID, assigned set number and the three assigned instructions (highlighted in yellow color)
- Attach your entity code (all modules), testbench code and output visuals (waveform with PC, ACC, Z and R if used)
- Challenges faced during the experiment, and conclusion

Project Report

Mini 8-bit Accumulator CPU in VHDL

CSE 450 (VLSI Lab) | Final Project | Spring 2026

1. Student Information and Assigned Instruction Set

Item	Details
Student Name	Tasnim Abrar Sajin
Student ID	22201119 (last digit = 9)
Assigned set	**Set 9**
Mandatory instructions	LDI, ADD
Assigned instructions	**NOT, SHL, JZ**
Complete instruction set	LDI, ADD, NOT, SHL, JZ
Language / tools	VHDL (IEEE.STD_LOGIC_1164, IEEE.NUMERIC_STD), Xilinx ISE project (XST / ISim). Waveforms in this report are from the testbench runs of the final code.

The CPU implements five instructions, all single-cycle with a 12-bit format (opcode[11:8] and operand[7:0]). The three instructions assigned to Set 9 are highlighted in Table 1.

Table 1: *Instruction set implemented by this CPU (Set 9)*

Mnemonic	Opcode	Operation	Flags
LDI imm	0001	ACC <- imm	Z updated
ADD imm	0010	ACC <- ACC + imm (8-bit, carry discarded)	Z updated
NOT	0111	ACC <- NOT ACC (operand ignored)	Z updated
SHL	1000	ACC <- ACC shifted left by 1, 0 enters LSB (operand ignored)	Z updated
JZ addr	1100	If Z = 1 then PC <- operand[3:0], else PC <- PC + 1	Z unchanged

2. Design Overview

The processor was designed bottom-up. Every logic function is derived from a single 2-input NAND gate; the gates are combined into adders, an 8-bit adder/subtractor (ALU), a zero detector, registers and finally the complete CPU. Each module has its own testbench, and the top-level testbench runs the Set 9 program of Appendix B and checks PC, ACC and Z after every clock edge with assertions.

2.1 Module hierarchy

Table 2: *Module hierarchy (lowest level first)*

Level	Module	Built from	Role
0	nand_gate	Dataflow: Y <= A nand B	Only primitive gate
1	not_gate, and_gate, or_gate, xor_gate	NAND gates only	Basic logic
2	half_adder, full_adder, mux_8bit	XOR / AND / OR / NOT gates	Arithmetic and selection
3	full_adder_8bit, register_8bit	8 full adders (ripple); D flip-flop process	8-bit adder, 8-bit register
4	alu_8bit, zero_flag, accumulator_8bit, program_counter	Level 1 to 3 modules	Datapath blocks

Level	Module	Built from	Role
4	program_rom, instruction_decoder, control_unit	Behavioural (case statements)	Program storage and control
5	cpu_top	All of the above	Complete CPU

2.2 CPU architecture

Figure 1 shows how the blocks of the final design are connected. The Program Counter addresses the Program ROM; the 12-bit instruction goes to the Instruction Decoder, which splits it into opcode and operand and raises one enable line (LOAD, ADD, NOT, SHL or JZ). The ALU adds the operand to ACC, the ACC next-value block selects the value to be written for the current opcode, and the Accumulator stores it on the rising clock edge. The Zero Flag watches ACC. The Next-PC logic selects PC + 1, the JZ target (when JZ is decoded and Z = 1) or 0 (reset).

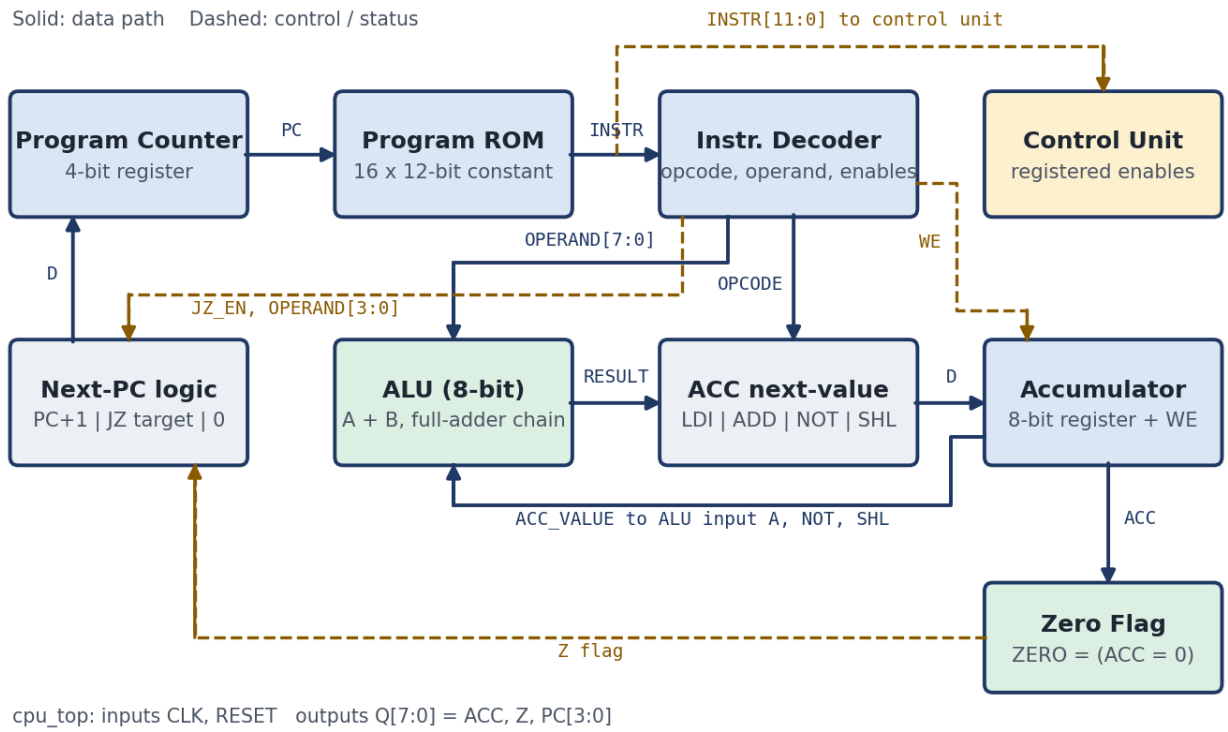


Figure 1: *Block diagram of the implemented 8-bit accumulator CPU (Set 9)*

2.3 Program stored in the ROM

The Program ROM holds exactly the Appendix B program of Set 9. Unused locations return 0x000, which is decoded as NOP.

Table 3: *Program ROM contents (Set 9)*

Address	ROM word	Instruction	Encoding (opcode & operand)
0	0x1FF	LDI 0xFF	0001 & 11111111
1	0x700	**NOT**	0111 & 00000000
2	0xC05	**JZ 0x05**	1100 & 00000101
3	0x277	ADD 0x77	0010 & 01110111
4	0x277	ADD 0x77	0010 & 01110111
5	0x211	ADD 0x11	0010 & 00010001
6	0x800	**SHL**	1000 & 00000000
7	0xC00	**JZ 0x00**	1100 & 00000000

Address	ROM word	Instruction	Encoding (opcode & operand)
8	0x800	SHL	1000 & 00000000

3. Basic Building Blocks

These small blocks are the foundation of the datapath. They are verified by their own testbenches (no assertion failures), so only their code is shown here.

3.1 NAND gate

The only primitive of the design. All other gates are made from it.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity nand_gate is
    Port (
        A : in  STD_LOGIC;
        B : in  STD_LOGIC;
        Y : out STD_LOGIC
    );
end nand_gate;
architecture Dataflow of nand_gate is
begin
    Y <= A nand B;
end Dataflow;
```

3.2 NOT gate

A NAND gate with both inputs tied together: Y = NOT(A).

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity not_gate is
    Port (
        A : in  STD_LOGIC;
        Y : out STD_LOGIC
    );
end not_gate;
architecture Structural of not_gate is
    -- component declarations omitted: nand_gate
begin
    NAND1: nand_gate port map (A => A, B => A, Y => Y);
end Structural;
```

3.3 AND gate

NAND followed by a NAND used as an inverter.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity and_gate is
    Port (
        A : in  STD_LOGIC;
        B : in  STD_LOGIC;
        Y : out STD_LOGIC
    );
end and_gate;
architecture Structural of and_gate is
    -- component declarations omitted: nand_gate
    signal nand_out : STD_LOGIC;
begin
    NAND1: nand_gate port map (A => A, B => B, Y => nand_out);
    NAND2: nand_gate port map (A => nand_out, B => nand_out, Y => Y);
end Structural;
```

3.4 OR gate

Two inverted inputs into a NAND: Y = A OR B (De Morgan).

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity or_gate is
    Port (
        A : in  STD_LOGIC;
        B : in  STD_LOGIC;
        Y : out STD_LOGIC
    );
end or_gate;
architecture Structural of or_gate is
    -- component declarations omitted: nand_gate
     signal not_A : STD_LOGIC;
```

```
    signal not_B : STD_LOGIC;
begin
    NAND1: nand_gate port map (A => A, B => A, Y => not_A);
    NAND2: nand_gate port map (A => B, B => B, Y => not_B);
    NAND3: nand_gate port map (A => not_A, B => not_B, Y => Y);
end Structural;
```

3.5 XOR gate

Four-NAND XOR structure.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity xor_gate is
    Port (
        A : in  STD_LOGIC;
        B : in  STD_LOGIC;
        Y : out STD_LOGIC
    );
end xor_gate;
architecture Structural of xor_gate is
    -- component declarations omitted: nand_gate
    signal nand1_out : STD_LOGIC;
    signal nand2_out : STD_LOGIC;
    signal nand3_out : STD_LOGIC;
begin
    NAND1: nand_gate port map (A => A, B => B, Y => nand1_out);
    NAND2: nand_gate port map (A => A, B => nand1_out, Y => nand2_out);
    NAND3: nand_gate port map (A => B, B => nand1_out, Y => nand3_out);
    NAND4: nand_gate port map (A => nand2_out, B => nand3_out, Y => Y);
end Structural;
```

3.6 Half adder

SUM = A XOR B and CARRY = A AND B.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity half_adder is
    Port (
        A     : in  STD_LOGIC;
        B     : in  STD_LOGIC;
        SUM   : out STD_LOGIC;
        CARRY : out STD_LOGIC
    );
end half_adder;
architecture Structural of half_adder is
    -- component declarations omitted: xor_gate, and_gate
begin
    XOR1: xor_gate port map (A => A, B => B, Y => SUM);
    AND1: and_gate port map (A => A, B => B, Y => CARRY);
end Structural;
```

3.7 Full adder

Two half adders and an OR gate for the carry.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity full_adder is
    Port (
        A    : in  STD_LOGIC;
        B    : in  STD_LOGIC;
        CIN  : in  STD_LOGIC;
        SUM  : out STD_LOGIC;
        COUT : out STD_LOGIC
    );
end full_adder;
architecture Structural of full_adder is
    -- component declarations omitted: half_adder, or_gate
    signal sum1  : STD_LOGIC;
    signal carry1 : STD_LOGIC;
    signal carry2 : STD_LOGIC;
begin
    HA1: half_adder port map (A => A, B => B, SUM => sum1, CARRY => carry1);
    HA2: half_adder port map (A => sum1, B => CIN, SUM => SUM, CARRY => carry2);
    OR1: or_gate port map (A => carry1, B => carry2, Y => COUT);
end Structural;
```

3.8 8-bit ripple-carry adder

Eight full adders chained through the carry vector C. This is the core of the ALU. (FA2 to FA6 follow the same pattern and are omitted here.)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity full_adder_8bit is
    Port (
        A    : in  STD_LOGIC_VECTOR(7 downto 0);
        B    : in  STD_LOGIC_VECTOR(7 downto 0);
        CIN  : in  STD_LOGIC;
        SUM  : out STD_LOGIC_VECTOR(7 downto 0);
        COUT : out STD_LOGIC
    );
end full_adder_8bit;
architecture Structural of full_adder_8bit is
    -- component declarations omitted: full_adder
    signal C : STD_LOGIC_VECTOR(8 downto 0);
begin
    C(0) <= CIN;
    FA0: full_adder port map (A => A(0), B => B(0), CIN => C(0), SUM => SUM(0), COUT => C(1));
    FA1: full_adder port map (A => A(1), B => B(1), CIN => C(1), SUM => SUM(1), COUT => C(2));
    -- FA2 ... FA6 : same pattern (A(i), B(i), C(i) -> SUM(i), C(i+1))
    FA7: full_adder port map (A => A(7), B => B(7), CIN => C(7), SUM => SUM(7), COUT => C(8));
    COUT <= C(8);
end Structural;
```

3.9 8-bit register

Eight D flip-flops with synchronous, active-high reset on the rising clock edge. Used by the PC and the accumulator.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity register_8bit is
    Port (
        D     : in  STD_LOGIC_VECTOR(7 downto 0);
        CLK   : in  STD_LOGIC;
        RESET : in  STD_LOGIC;
        Q     : out STD_LOGIC_VECTOR(7 downto 0)
    );
end register_8bit;
architecture Structural of register_8bit is

    signal REG_DATA : STD_LOGIC_VECTOR(7 downto 0);
    signal REG_NEXT : STD_LOGIC_VECTOR(7 downto 0);
    signal CLK_NOT  : STD_LOGIC;

begin
    -- Invert the clock using your NOT gate
    CLK_INV : entity work.not_gate
        port map (
            A => CLK,
            Y => CLK_NOT
        );
    -- Feedback mux: select zero during reset, otherwise D
    -- Assumes mux A is selected for SEL='0'
    -- and mux B for SEL='1'.
    REG_MUX : entity work.mux_8bit
        port map (
            A   => (others => '0'),
            B   => D,
            SEL => RESET,
            Y   => REG_NEXT
        );
    -- Eight storage bits
    REG_BITS : for i in 0 to 7 generate
        BIT_REG : entity work.d_flip_flop
            port map (
                D     => REG_NEXT(i),
                CLK   => CLK,
                RESET => '0',
                Q     => REG_DATA(i)
            );
    end generate;

    Q <= REG_DATA;

end Structural;
```

3.10 8-bit 2-to-1 multiplexer

Gate-level multiplexer (SEL = 0 selects A, SEL = 1 selects B). It is used as the hold/load selector in the accumulator.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity d_flip_flop is
    Port (
        D     : in  STD_LOGIC;
        CLK   : in  STD_LOGIC;
        RESET : in  STD_LOGIC;
        Q     : out STD_LOGIC
    );
end d_flip_flop;
architecture Structural of d_flip_flop is

    component d_latch
```

```
        Port (
            D   : in  STD_LOGIC;
            EN  : in  STD_LOGIC;
            Q   : out STD_LOGIC
        );
    end component;
    signal CLK_NOT : STD_LOGIC;
    signal MASTER_Q : STD_LOGIC;
    signal SLAVE_Q  : STD_LOGIC;

begin

    -- Invert clock
    CLK_INV : entity work.not_gate
        port map (
            A => CLK,
            Y => CLK_NOT
        );
    -- Master latch enabled when CLK = 0
    MASTER_LATCH : d_latch
        port map (
            D  => D,
            EN => CLK_NOT,
            Q  => MASTER_Q
        );

    -- Slave latch enabled when CLK = 1
    SLAVE_LATCH : d_latch
        port map (
            D  => MASTER_Q,
            EN => CLK,
            Q  => SLAVE_Q
        );
   -- Synchronous reset
    process(CLK)
    begin
        if rising_edge(CLK) then
            if RESET = '1' then
                -- Reset is handled at the output below
                null;
            end if;
        end if;
    end process;

    Q <= SLAVE_Q when RESET = '0' else '0';

end Structural;
```

4. Main Components

Each main component is described below with its block symbol, VHDL code, the stimulus used in its testbench and the simulated waveform.

4.1 ALU (alu_8bit)

The ALU is a combinational 8-bit adder/subtractor. For OP = 0 it computes RESULT = A + B. For OP = 1 the bits of B are inverted and OP is used as carry-in, giving A + NOT(B) + 1 = A - B. COUT is the carry for an addition and the borrow for a subtraction. In the CPU the ALU performs the ADD instruction (OP tied to 0); the subtract path is implemented and tested but not needed by Set 9.

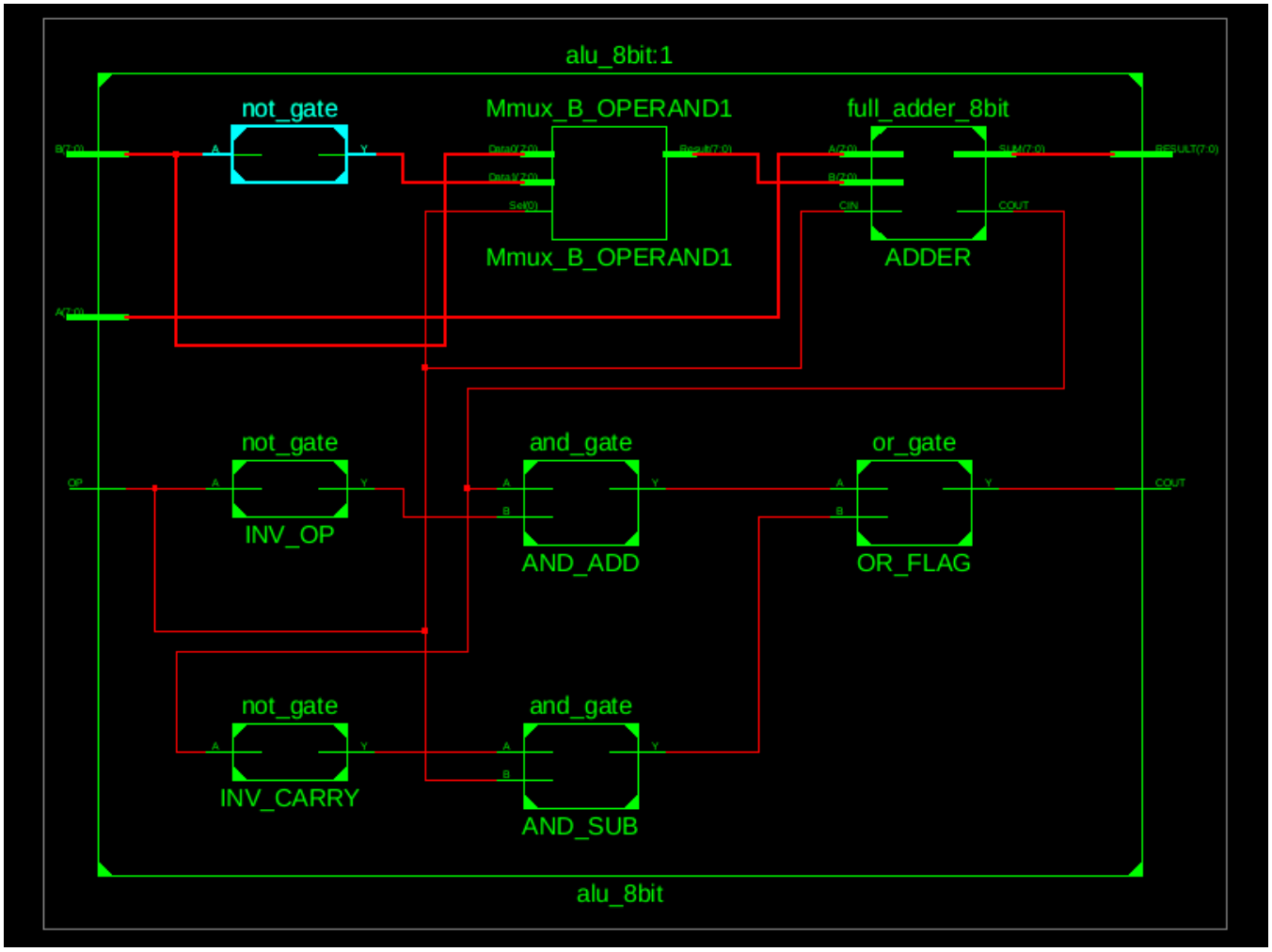


Figure 2: *Block symbol of alu_8bit*

VHDL code: alu_8bit.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity alu_8bit is
    Port (
        A      : in  STD_LOGIC_VECTOR(7 downto 0);
        B      : in  STD_LOGIC_VECTOR(7 downto 0);
        OP     : in  STD_LOGIC;
        RESULT : out STD_LOGIC_VECTOR(7 downto 0);
        COUT   : out STD_LOGIC
    );
end alu_8bit;
architecture Structural of alu_8bit is

    component full_adder_8bit
        Port (
            A    : in  STD_LOGIC_VECTOR(7 downto 0);
            B    : in  STD_LOGIC_VECTOR(7 downto 0);
            CIN  : in  STD_LOGIC;
            SUM  : out STD_LOGIC_VECTOR(7 downto 0);
            COUT : out STD_LOGIC
        );
    end component;

    component not_gate
        Port (
            A : in  STD_LOGIC;
            Y : out STD_LOGIC
        );
    end component;
    component and_gate
        Port (
            A : in  STD_LOGIC;
            B : in  STD_LOGIC;
            Y : out STD_LOGIC
        );
    end component;

    component or_gate
        Port (
```

```
            A : in  STD_LOGIC;
            B : in  STD_LOGIC;
            Y : out STD_LOGIC
        );
    end component;
    signal B_INV      : STD_LOGIC_VECTOR(7 downto 0);
    signal B_OPERAND  : STD_LOGIC_VECTOR(7 downto 0);
    signal SUM_INT    : STD_LOGIC_VECTOR(7 downto 0);
    signal CARRY_INT  : STD_LOGIC;
    signal BORROW_INT : STD_LOGIC;
    signal NOT_OP     : STD_LOGIC;
    signal FLAG_ADD   : STD_LOGIC;
    signal FLAG_SUB   : STD_LOGIC;

begin
    -- Invert every bit of B for subtraction
    GEN_NOT_B: for i in 0 to 7 generate
        INV_B: not_gate
            port map (
                A => B(i),
                Y => B_INV(i)
            );
    end generate GEN_NOT_B;

    -- Select B for addition or NOT B for subtraction
    GEN_OPERAND: for i in 0 to 7 generate
        B_OPERAND(i) <= B(i) when OP = '0'
                        else B_INV(i);
    end generate GEN_OPERAND;
    -- A + B for OP=0
    -- A + NOT(B) + 1 for OP=1
    ADDER: full_adder_8bit
        port map (
            A    => A,
            B    => B_OPERAND,
            CIN  => OP,
            SUM  => SUM_INT,
            COUT => CARRY_INT
        );

    -- Invert OP
    INV_OP: not_gate
        port map (
            A => OP,
            Y => NOT_OP
        );
    -- For subtraction, borrow = NOT(carry out)
    INV_CARRY: not_gate
        port map (
            A => CARRY_INT,
            Y => BORROW_INT
        );

    -- Select carry flag for addition
    AND_ADD: and_gate
        port map (
            A => CARRY_INT,
            B => NOT_OP,
            Y => FLAG_ADD
        );
    -- Select borrow flag for subtraction
    AND_SUB: and_gate
        port map (
            A => BORROW_INT,
            B => OP,
            Y => FLAG_SUB
        );

    -- Combine the selected flags
    OR_FLAG: or_gate
        port map (
            A => FLAG_ADD,
            B => FLAG_SUB,
            Y => COUT
        );

    RESULT <= SUM_INT;

end Structural;
```

Testbench stimulus: alu_8bit_tb.vhd (excerpt)

```
    -- Stimulus process
    stim_proc: process
    begin
         -- Test 1: Addition 10 + 20 = 30
         A <= "00001010";
         B <= "00010100";
         OP <= '0';
         wait for 20 ns;

         -- Test 2: Addition 255 + 1 = 256
         A <= "11111111";
         B <= "00000001";
         OP <= '0';
         wait for 20 ns;

         -- Test 3: Addition 100 + 50 = 150
         A <= "01100100";
         B <= "00110010";
         OP <= '0';
         wait for 20 ns;

         -- Test 4: Subtraction 20 - 10 = 10
         A <= "00010100";
         B <= "00001010";
         OP <= '1';
         wait for 20 ns;

         -- Test 5: Subtraction 5 - 9 = -4
         -- 8-bit result is 252 (11111100)
         A <= "00000101";
         B <= "00001001";
         OP <= '1';
         wait for 20 ns;

         -- Test 6: Subtraction 9 - 9 = 0
         A <= "00001001";
         B <= "00001001";
         OP <= '1';
         wait for 20 ns;

         -- Test 7: Subtraction 0 - 1 = -1
         -- 8-bit result is 255 (11111111)
         A <= "00000000";
         B <= "00000001";
         OP <= '1';
         wait for 20 ns;

         wait;
     end process;
```

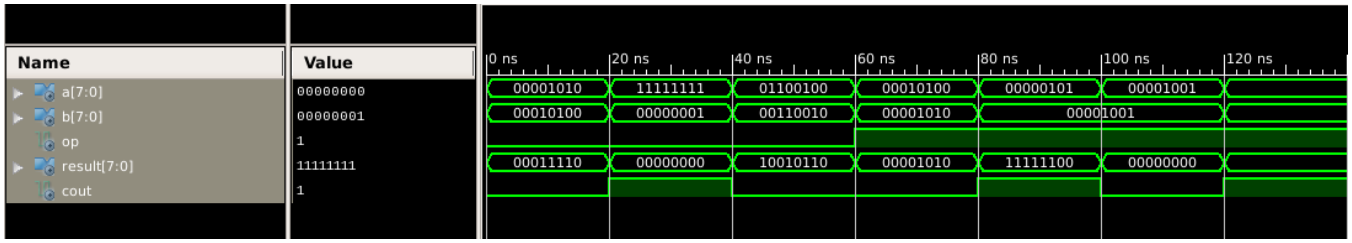


Figure 3: *Simulation waveform of alu_8bit*

The first three vectors use OP = 0: 0x0A + 0x14 = 0x1E, 0xFF + 0x01 = 0x00 with COUT = 1 (carry), and 0x64 + 0x32 = 0x96. The last four use OP = 1: 0x14 - 0x0A = 0x0A, 0x05 - 0x09 = 0xFC with COUT = 1 (borrow), 0x09 - 0x09 = 0x00 and 0x00 - 0x01 = 0xFF with borrow. All results are correct.

4.2 Zero flag (zero_flag)

The zero detector inverts all eight bits of its input and combines them with a tree of seven AND gates, so ZERO = 1 only when every bit of A is 0. It is connected to the accumulator output, so Z always reflects the current ACC value: it is 1 after reset and is updated whenever ACC is written by LDI, ADD, NOT or SHL.

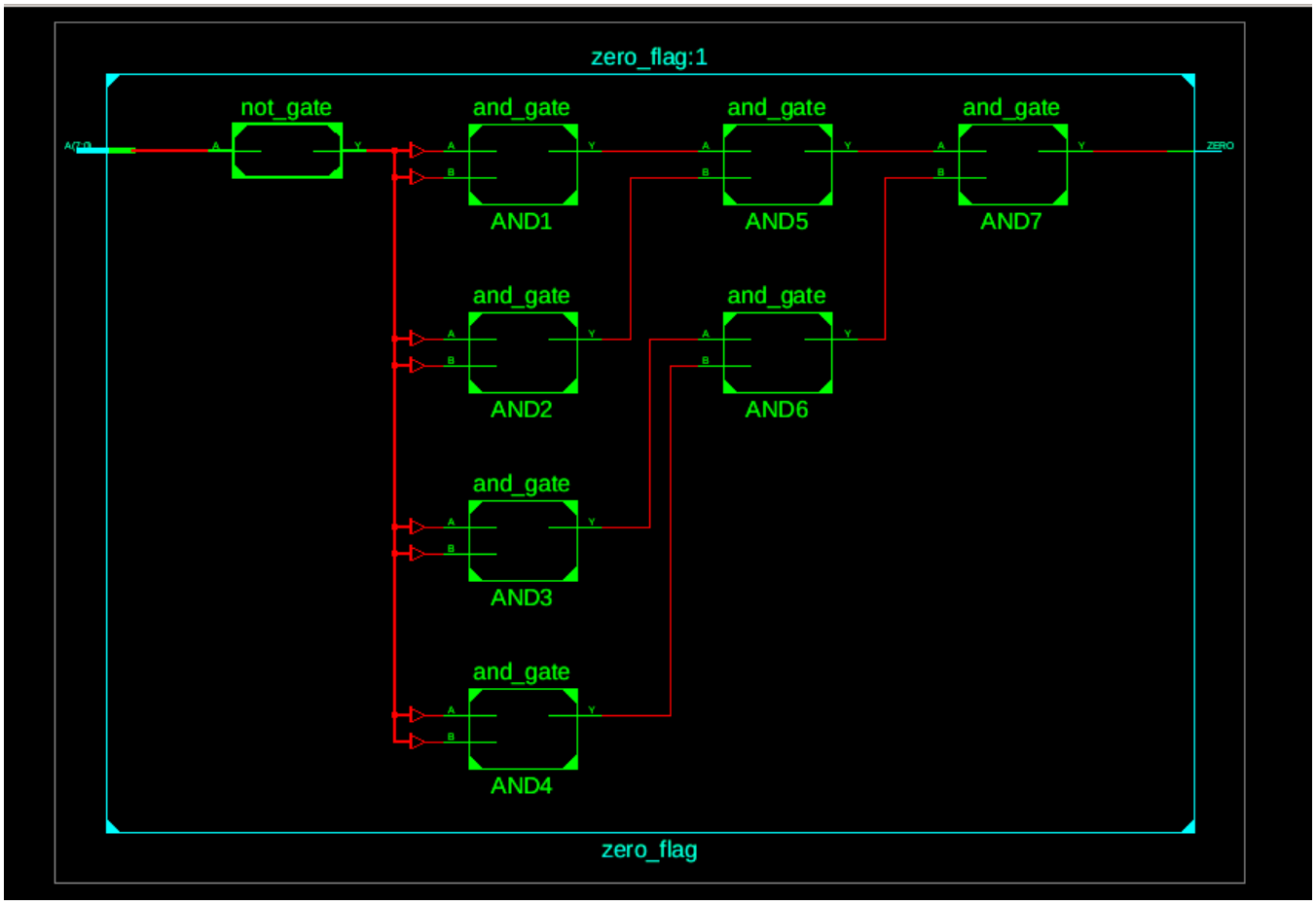


Figure 4: *Block symbol of zero_flag*

VHDL code: zero_flag.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity zero_flag is
    Port (
        A     : in  STD_LOGIC_VECTOR(7 downto 0);
        ZERO  : out STD_LOGIC
    );
end zero_flag;

architecture Structural of zero_flag is
    component not_gate
        Port (
            A : in  STD_LOGIC;
            Y : out STD_LOGIC
        );
    end component;

    component and_gate
        Port (
            A : in  STD_LOGIC;
            B : in  STD_LOGIC;
            Y : out STD_LOGIC
        );
    end component;
    signal N0, N1, N2, N3 : STD_LOGIC;
    signal N4, N5, N6, N7 : STD_LOGIC;
    signal G1, G2, G3, G4 : STD_LOGIC;
    signal G5, G6 : STD_LOGIC;

begin
    -- Invert every input bit
    INV0: not_gate port map (A => A(0), Y => N0);
    INV1: not_gate port map (A => A(1), Y => N1);
    INV2: not_gate port map (A => A(2), Y => N2);
    INV3: not_gate port map (A => A(3), Y => N3);
    INV4: not_gate port map (A => A(4), Y => N4);
    INV5: not_gate port map (A => A(5), Y => N5);
    INV6: not_gate port map (A => A(6), Y => N6);
    INV7: not_gate port map (A => A(7), Y => N7);
    -- Combine the inverted bits
    AND1: and_gate port map (A => N0, B => N1, Y => G1);
    AND2: and_gate port map (A => N2, B => N3, Y => G2);
    AND3: and_gate port map (A => N4, B => N5, Y => G3);
    AND4: and_gate port map (A => N6, B => N7, Y => G4);
```

```
    AND5: and_gate port map (A => G1, B => G2, Y => G5);
    AND6: and_gate port map (A => G3, B => G4, Y => G6);

    AND7: and_gate port map (A => G5, B => G6, Y => ZERO);

end Structural;
```

Testbench stimulus: zero_flag_tb.vhd (excerpt)

```
    STIMULUS: PROCESS
    BEGIN

        -- Test 1: All bits are zero
        A <= x"00";
        WAIT FOR 10 ns;
        ASSERT ZERO = '1'
            REPORT "Test 1 FAILED: Expected ZERO = 1"
            SEVERITY ERROR;
        -- Test 2: Input is 1
        A <= x"01";
        WAIT FOR 10 ns;
        ASSERT ZERO = '0'
            REPORT "Test 2 FAILED: Expected ZERO = 0"
            SEVERITY ERROR;

        -- Test 3: Input is FF
        A <= x"FF";
        WAIT FOR 10 ns;
        ASSERT ZERO = '0'
            REPORT "Test 3 FAILED: Expected ZERO = 0"
            SEVERITY ERROR;
        -- Test 4: Input is 80
        A <= x"80";
        WAIT FOR 10 ns;
        ASSERT ZERO = '0'
            REPORT "Test 4 FAILED: Expected ZERO = 0"
            SEVERITY ERROR;

        -- Test 5: Input is 0 again
        A <= x"00";
        WAIT FOR 10 ns;
        ASSERT ZERO = '1'
            REPORT "Test 5 FAILED: Expected ZERO = 1"
            SEVERITY ERROR;

        REPORT "All zero flag tests completed."
            SEVERITY NOTE;

        WAIT;
    END PROCESS;
```

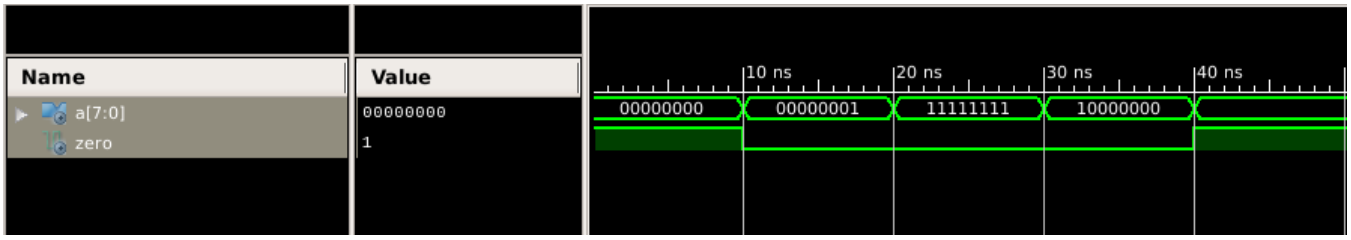


Figure 5: *Simulation waveform of zero_flag*

ZERO is 1 for A = 0x00 and drops to 0 for A = 0x01, 0xFF and 0x80. It returns to 1 when A = 0x00 again.

4.3 Accumulator (accumulator_8bit)

The accumulator is the main working register and the CPU output Q. It consists of an 8-bit register and a hold/load multiplexer. With WE = 0 the register keeps its value; with WE = 1 it loads D on the next rising clock edge. RESET is synchronous and active-high.

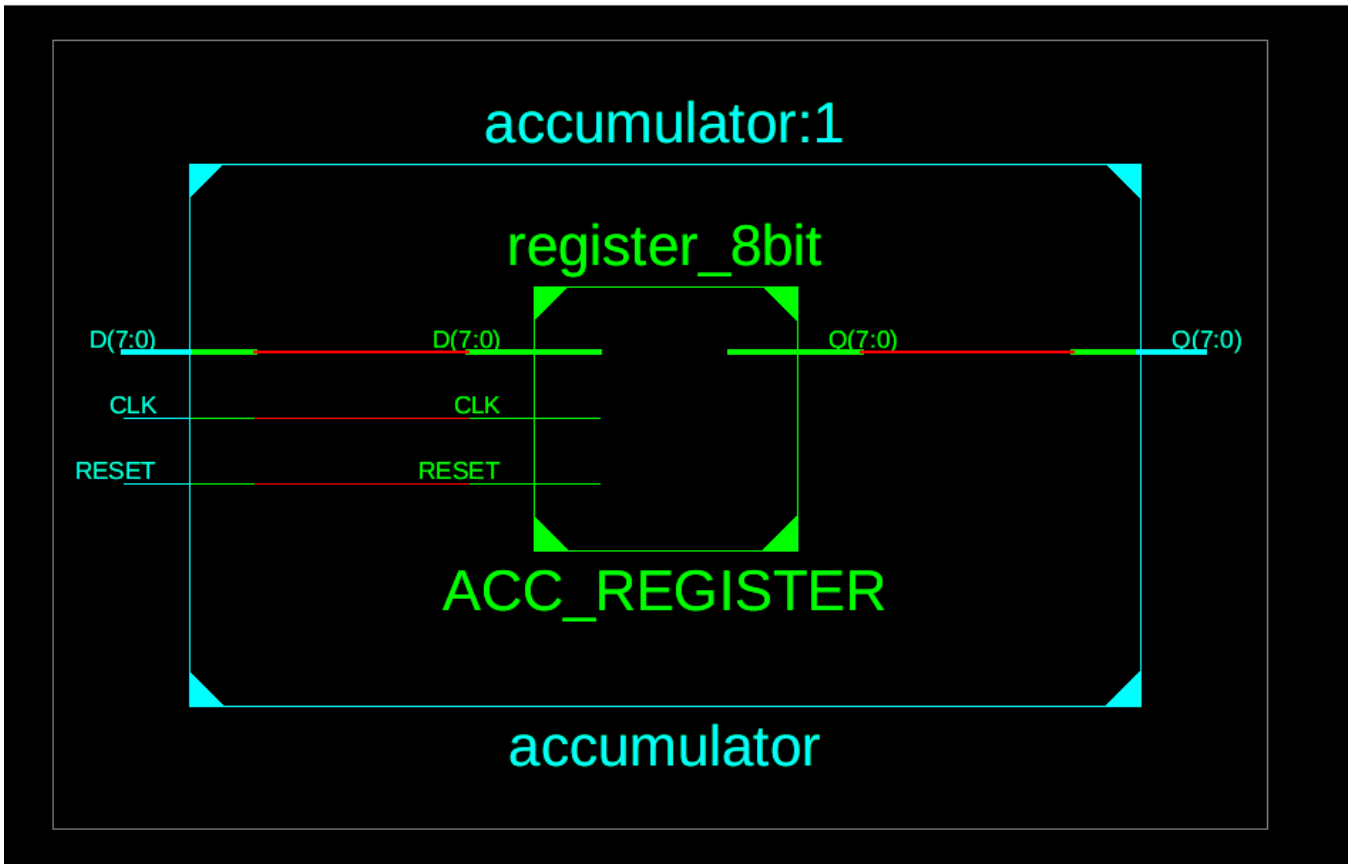


Figure 6: *Block symbol of accumulator_8bit*

VHDL code: accumulator_8bit.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity accumulator is
    Port (
        D     : in  STD_LOGIC_VECTOR(7 downto 0);
        CLK   : in  STD_LOGIC;
        RESET : in  STD_LOGIC;
        Q     : out STD_LOGIC_VECTOR(7 downto 0)
    );
end accumulator;
architecture Structural of accumulator is
begin
    ACC_REGISTER : entity work.register_8bit
        port map (
            D     => D,
            CLK   => CLK,
            RESET => RESET,
            Q     => Q
        );
```

```
end Structural;
```

Testbench stimulus: accumulator_8bit_tb.vhd (excerpt)

```
    STIMULUS: PROCESS
    BEGIN

        -- Test 1: Reset
        RESET <= '1';
        D <= x"00";
        WAIT FOR 25 ns;
        ASSERT Q = x"00"
            REPORT "Test 1 FAILED: Reset"
            SEVERITY ERROR;
        -- Test 2: Add 5
        RESET <= '0';
        D <= x"05";
        WAIT FOR 20 ns;
        ASSERT Q = x"05"
            REPORT "Test 2 FAILED: Expected 5"
            SEVERITY ERROR;

        -- Test 3: Add 10; total = 15
        D <= x"0A";
        WAIT FOR 20 ns;
        ASSERT Q = x"0F"
            REPORT "Test 3 FAILED: Expected 15"
            SEVERITY ERROR;
        -- Test 4: Add 20; total = 35
        D <= x"14";
        WAIT FOR 20 ns;
        ASSERT Q = x"23"
            REPORT "Test 4 FAILED: Expected 35"
            SEVERITY ERROR;

        -- Test 5: Add 255; 35 + 255 = 290
        -- 8-bit result = 34
        D <= x"FF";
        WAIT FOR 20 ns;
        ASSERT Q = x"22"
            REPORT "Test 5 FAILED: Expected 34"
            SEVERITY ERROR;
        -- Test 6: Reset again
        RESET <= '1';
        D <= x"00";
        WAIT FOR 20 ns;
        ASSERT Q = x"00"
            REPORT "Test 6 FAILED: Reset"
            SEVERITY ERROR;

        REPORT "All accumulator tests completed."
            SEVERITY NOTE;

        WAIT;
    END PROCESS;
```

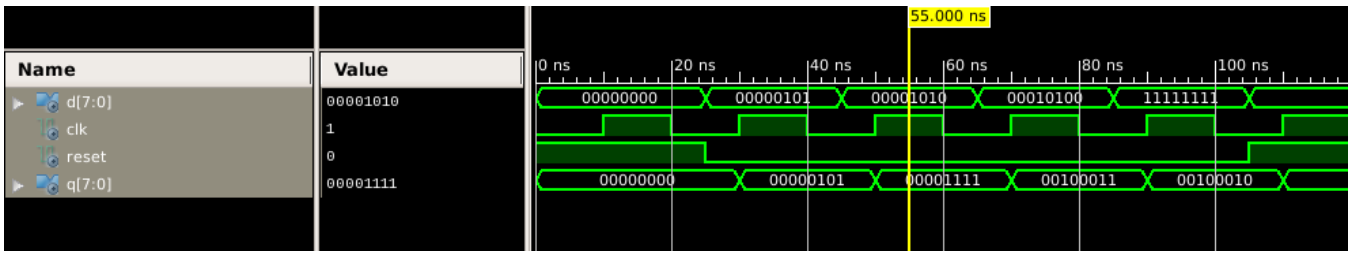


Figure 7: *Simulation waveform of accumulator_8bit*

After reset Q = 0x00. At the next rising edge with WE = 1 the value 0x5A is loaded. When D changes to 0xFF while WE = 0, Q stays 0x5A (hold). With WE = 1 again, 0xC3 is loaded, and the final reset clears Q to 0x00.

4.4 Program counter (program_counter)

The program counter is a 4-bit register (the low nibble of an 8-bit register) that holds the address of the current instruction. Its input D is the next address computed in cpu_top: PC + 1 normally, or the JZ target when a jump is taken. A synchronous reset sets PC to 0.

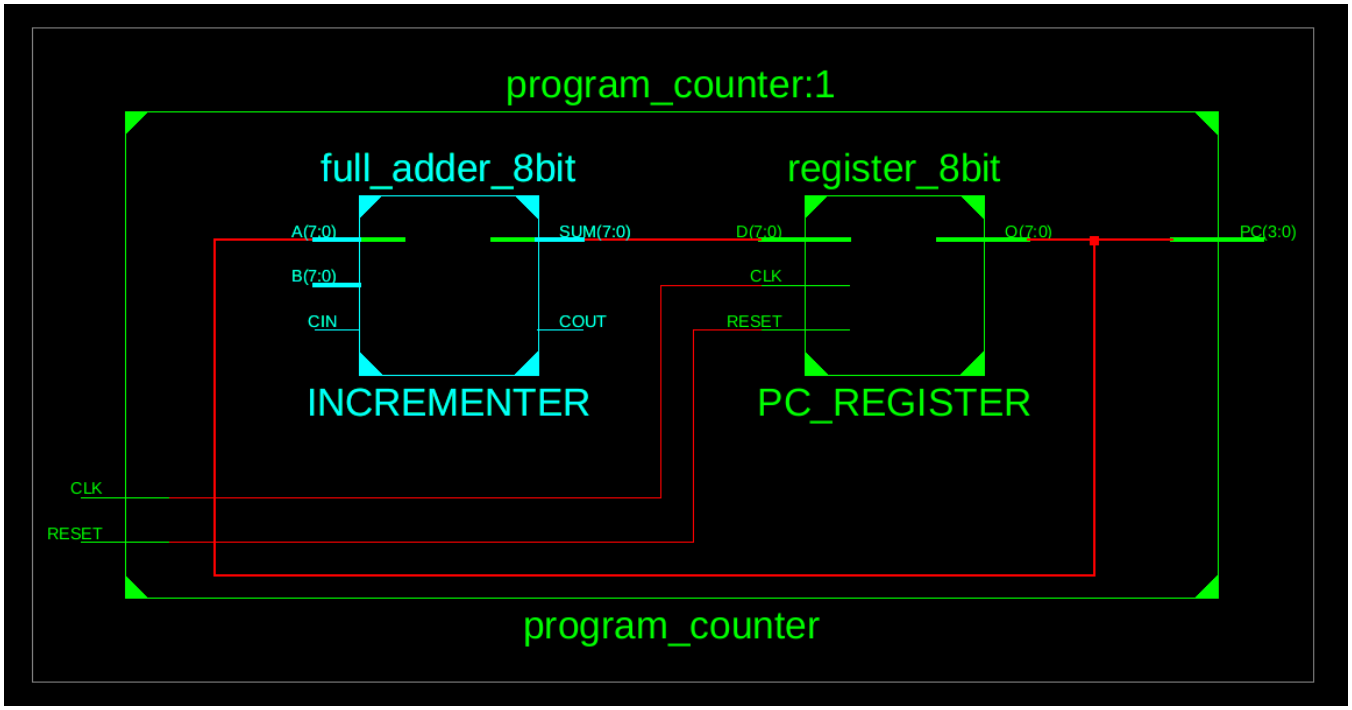


Figure 8: *Block symbol of program_counter*

VHDL code: program_counter.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity program_counter is
    Port (
        CLK   : in  STD_LOGIC;
        RESET : in  STD_LOGIC;
        LOAD  : in  STD_LOGIC;
        D     : in  STD_LOGIC_VECTOR(3 downto 0);
        PC    : out STD_LOGIC_VECTOR(3 downto 0)
    );
end program_counter;
architecture Structural of program_counter is

    component full_adder_8bit
        Port (
            A    : in  STD_LOGIC_VECTOR(7 downto 0);
            B    : in  STD_LOGIC_VECTOR(7 downto 0);
            CIN  : in  STD_LOGIC;
            SUM  : out STD_LOGIC_VECTOR(7 downto 0);
            COUT : out STD_LOGIC
        );
    end component;
    component register_8bit
        Port (
            D     : in  STD_LOGIC_VECTOR(7 downto 0);
            CLK   : in  STD_LOGIC;
            RESET : in  STD_LOGIC;
            Q     : out STD_LOGIC_VECTOR(7 downto 0)
        );
    end component;

    signal PC_CURRENT : STD_LOGIC_VECTOR(7 downto 0);
    signal PC_NEXT    : STD_LOGIC_VECTOR(7 downto 0);
    signal CARRY_OUT  : STD_LOGIC;
begin

    INCREMENTER: full_adder_8bit
        port map (
            A    => PC_CURRENT,
            B    => "00000001",
            CIN  => '0',
            SUM  => PC_NEXT,
            COUT => CARRY_OUT
        );
    PC_REGISTER: register_8bit
        port map (
            D     => PC_NEXT,
            CLK   => CLK,
            RESET => RESET,
            Q     => PC_CURRENT
        );

    PC <= PC_CURRENT(3 downto 0);

end Structural;
```

Testbench stimulus: program_counter_tb.vhd (excerpt)

```
    STIMULUS: PROCESS
    BEGIN
        -- Test 1: Reset PC to zero
        RESET <= '1';
        LOAD  <= '0';
        WAIT UNTIL rising_edge(CLK);
        WAIT FOR 1 ns;

        ASSERT PC = "0000"
            REPORT "Test 1 FAILED: PC should be zero"
            SEVERITY ERROR;

        -- Test 2: First increment
        RESET <= '0';

        WAIT UNTIL rising_edge(CLK);
        WAIT FOR 1 ns;

        ASSERT PC = "0001"
            REPORT "Test 2 FAILED: PC should be 1"
            SEVERITY ERROR;
        -- Test 3: Second increment
        WAIT UNTIL rising_edge(CLK);
        WAIT FOR 1 ns;

        ASSERT PC = "0010"
            REPORT "Test 3 FAILED: PC should be 2"
            SEVERITY ERROR;

        -- Test 4: Third increment
        WAIT UNTIL rising_edge(CLK);
        WAIT FOR 1 ns;

        ASSERT PC = "0011"
            REPORT "Test 4 FAILED: PC should be 3"
            SEVERITY ERROR;
        -- Test 5: Load jump address 7
        D    <= "0111";
        LOAD <= '1';

        WAIT UNTIL rising_edge(CLK);
        WAIT FOR 1 ns;

        ASSERT PC = "0111"
            REPORT "Test 5 FAILED: PC should load address 7"
            SEVERITY ERROR;

        -- Test 6: Increment after jump
        LOAD <= '0';

        WAIT UNTIL rising_edge(CLK);
        WAIT FOR 1 ns;
        ASSERT PC = "1000"
            REPORT "Test 6 FAILED: PC should increment to 8"
            SEVERITY ERROR;

        -- Test 7: Reset again
        RESET <= '1';

        WAIT UNTIL rising_edge(CLK);
        WAIT FOR 1 ns;

        ASSERT PC = "0000"
            REPORT "Test 7 FAILED: PC did not reset"
            SEVERITY ERROR;

        REPORT "All program counter tests completed."
            SEVERITY NOTE;
        WAIT;
    END PROCESS;
```

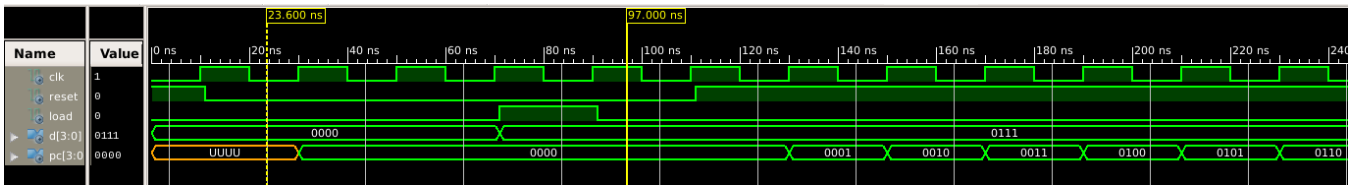


Figure 9: *Simulation waveform of program_counter*

PC is 0 during reset, then follows D on every rising edge: 1, 2 and then 5 (a jump-style load). The last reset returns PC to 0.

4.5 Program ROM (program_rom)

The ROM is a 16 x 12-bit constant array written as a VHDL case statement. It is read combinationally with PC as address (no clock). It contains the nine instructions of Table 3; every other address returns 0x000 (NOP).

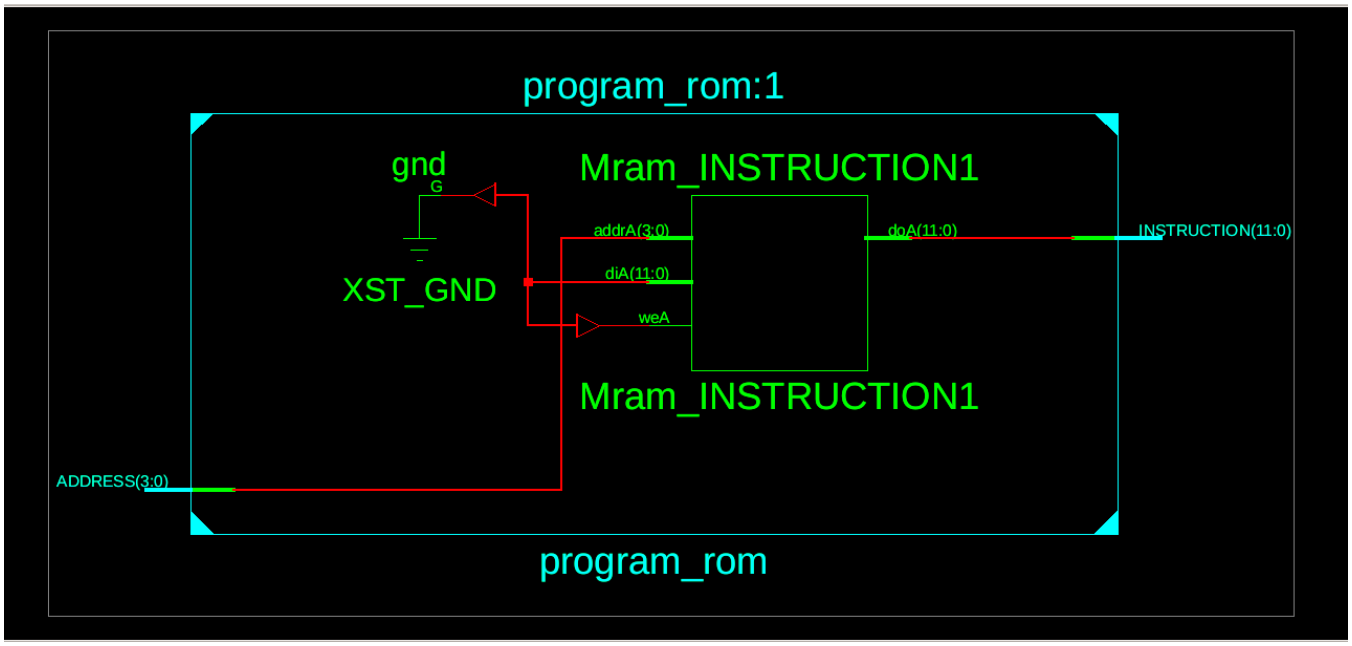


Figure 10: *Block symbol of program_rom*

VHDL code: program_rom.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity program_rom is
    Port (
        ADDRESS     : in  STD_LOGIC_VECTOR(3 downto 0);
        INSTRUCTION : out STD_LOGIC_VECTOR(11 downto 0)
    );
end program_rom;
architecture Behavioral of program_rom is
begin
    process(ADDRESS)
    begin
        case ADDRESS is
            when "0000" => INSTRUCTION <= x"1FF"; -- LDI FF
            when "0001" => INSTRUCTION <= x"700"; -- NOT
            when "0010" => INSTRUCTION <= x"C05"; -- JZ 05
            when "0011" => INSTRUCTION <= x"277"; -- ADD 77
            when "0100" => INSTRUCTION <= x"277"; -- ADD 77
            when "0101" => INSTRUCTION <= x"211"; -- ADD 11
            when "0110" => INSTRUCTION <= x"800"; -- SHL
            when "0111" => INSTRUCTION <= x"C00"; -- JZ 00
            when "1000" => INSTRUCTION <= x"800"; -- SHL
            when others => INSTRUCTION <= x"000"; -- NOP
        end case;
    end process;
end Behavioral;
```

Testbench stimulus: program_rom_tb.vhd (excerpt)

```
    STIMULUS: process
    begin
        ADDRESS <= x"0";
        wait for 10 ns;
        assert INSTRUCTION = x"1FF"
            report "Address 0 failed"
            severity error;

        ADDRESS <= x"1";
        wait for 10 ns;
        assert INSTRUCTION = x"700"
            report "Address 1 failed"
            severity error;
        ADDRESS <= x"2";
        wait for 10 ns;
        assert INSTRUCTION = x"C05"
            report "Address 2 failed"
            severity error;

        ADDRESS <= x"3";
        wait for 10 ns;
        assert INSTRUCTION = x"277"
            report "Address 3 failed"
            severity error;
        ADDRESS <= x"4";
        wait for 10 ns;
        assert INSTRUCTION = x"277"
            report "Address 4 failed"
            severity error;

        ADDRESS <= x"5";
        wait for 10 ns;
        assert INSTRUCTION = x"211"
            report "Address 5 failed"
            severity error;
        ADDRESS <= x"6";
        wait for 10 ns;
        assert INSTRUCTION = x"800"
            report "Address 6 failed"
            severity error;

        ADDRESS <= x"7";
        wait for 10 ns;
        assert INSTRUCTION = x"C00"
            report "Address 7 failed"
            severity error;
        ADDRESS <= x"8";
        wait for 10 ns;
        assert INSTRUCTION = x"800"
            report "Address 8 failed"
            severity error;

        ADDRESS <= x"9";
        wait for 10 ns;
        assert INSTRUCTION = x"000"
            report "Unused address should return NOP"
            severity error;

        report "ROM tests completed."
            severity note;

        wait;
    end process;
```

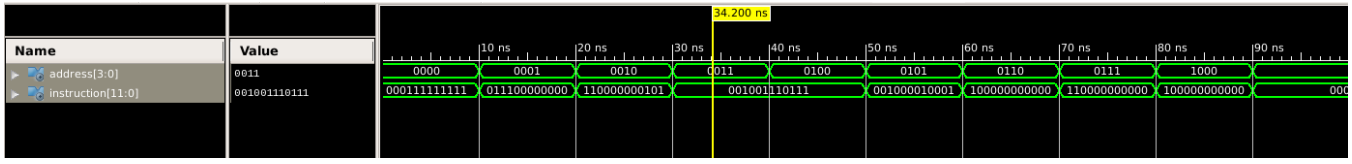


Figure 11: *Simulation waveform of program_rom*

Stepping the address from 0 to 9 gives 0x1FF, 0x700, 0xC05, 0x277, 0x277, 0x211, 0x800, 0xC00, 0x800 and 0x000 for the unused address 9. The output changes immediately with the address, which confirms the combinational read.

4.6 Instruction decoder (instruction_decoder)

The decoder splits the 12-bit instruction into OPCODE (bits 11:8) and OPERAND (bits 7:0) and raises exactly one enable line for the five supported opcodes: LOAD_EN (LDI), ADD_EN, NOT_EN, SHL_EN or JZ_EN. Any other opcode, including 0000, raises no enable and acts as a NOP.

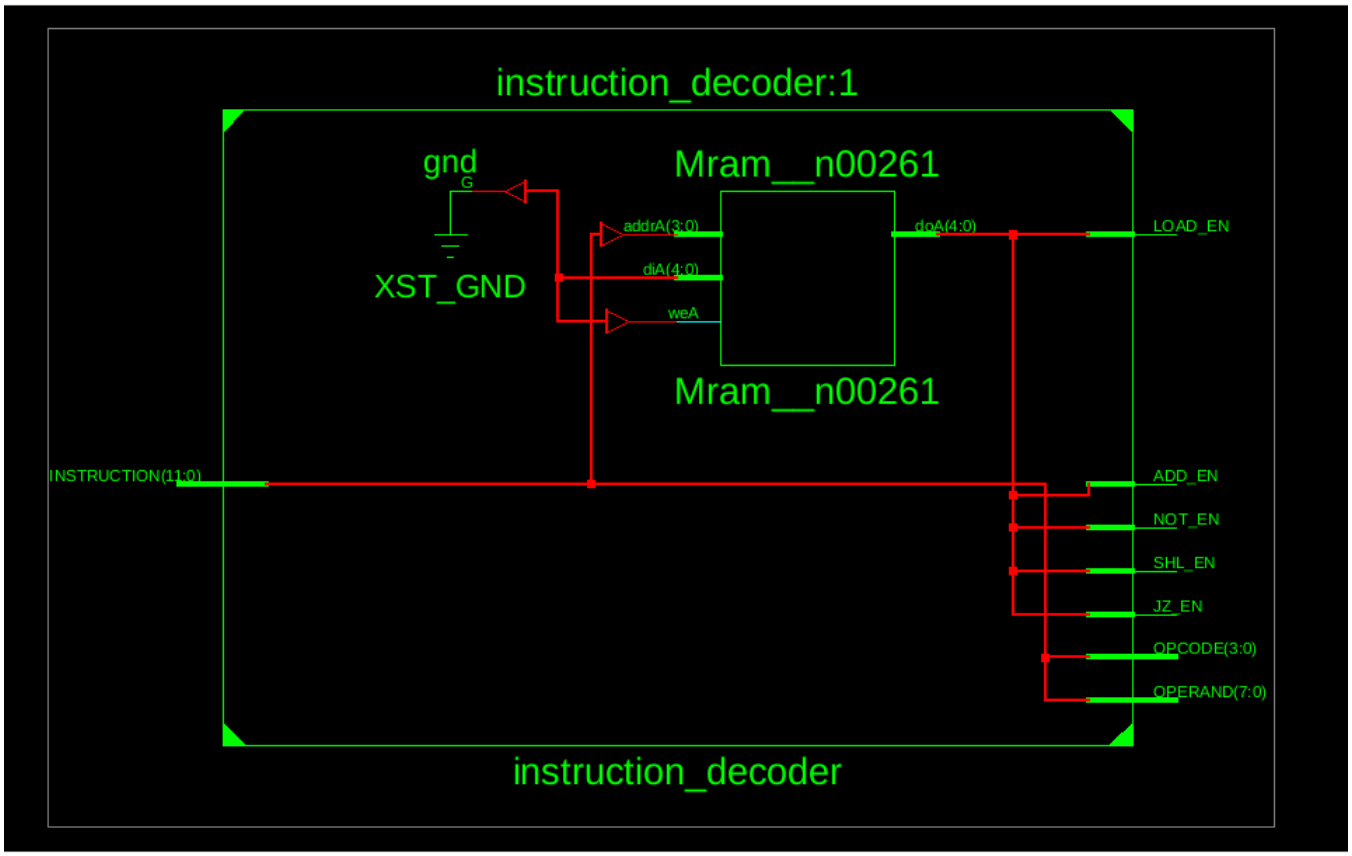


Figure 12: *Block symbol of instruction_decoder*

VHDL code: instruction_decoder.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity instruction_decoder is
    Port (
        INSTRUCTION : in  STD_LOGIC_VECTOR(11 downto 0);
        OPCODE      : out STD_LOGIC_VECTOR(3 downto 0);
        OPERAND     : out STD_LOGIC_VECTOR(7 downto 0);
        LOAD_EN     : out STD_LOGIC;
        ADD_EN      : out STD_LOGIC;
        NOT_EN      : out STD_LOGIC;
        SHL_EN      : out STD_LOGIC;
        JZ_EN       : out STD_LOGIC
    );
end instruction_decoder;
architecture Behavioral of instruction_decoder is
begin

    process(INSTRUCTION)
    begin
        OPCODE  <= INSTRUCTION(11 downto 8);
        OPERAND <= INSTRUCTION(7 downto 0);

        LOAD_EN <= '0';
        ADD_EN  <= '0';
        NOT_EN  <= '0';
        SHL_EN  <= '0';
        JZ_EN   <= '0';
        case INSTRUCTION(11 downto 8) is

            when "0001" =>
                LOAD_EN <= '1'; -- LDI

            when "0010" =>
                ADD_EN <= '1';  -- ADD

            when "0111" =>
                NOT_EN <= '1';  -- NOT
            when "1000" =>
                SHL_EN <= '1';  -- SHL

            when "1100" =>
                JZ_EN <= '1';   -- JZ

            when others =>
                null;           -- NOP

        end case;
    end process;
```

```
end Behavioral;
```

Testbench stimulus: instruction_decoder_tb.vhd (excerpt)

```
    stimulus: process
    begin

        -- Test LDI instruction
        INSTRUCTION <= x"1FF";
        wait for 10 ns;
        assert LOAD_EN = '1' and OPERAND = x"FF"
            report "LDI decode failed"
            severity error;
        -- Test ADD instruction
        INSTRUCTION <= x"277";
        wait for 10 ns;
        assert ADD_EN = '1' and OPERAND = x"77"
            report "ADD decode failed"
            severity error;

        -- Test NOT instruction
        INSTRUCTION <= x"700";
        wait for 10 ns;
        assert NOT_EN = '1'
            report "NOT decode failed"
            severity error;
        -- Test SHL instruction
        INSTRUCTION <= x"800";
        wait for 10 ns;
        assert SHL_EN = '1'
            report "SHL decode failed"
            severity error;

        -- Test JZ instruction
        INSTRUCTION <= x"C05";
        wait for 10 ns;
        assert JZ_EN = '1' and OPERAND = x"05"
            report "JZ decode failed"
            severity error;

        -- Test NOP instruction
        INSTRUCTION <= x"000";
        wait for 10 ns;
        assert LOAD_EN = '0' and
               ADD_EN  = '0' and
               NOT_EN  = '0' and
               SHL_EN  = '0' and
               JZ_EN   = '0'
            report "NOP decode failed"
            severity error;

        report "Instruction decoder test completed successfully."
            severity note;

        wait;
    end process;
```



Figure 13: *Simulation waveform of instruction_decoder*

0x1FF raises LOAD_EN, 0x277 raises ADD_EN, 0x700 raises NOT_EN, 0x800 raises SHL_EN and 0xC05 raises JZ_EN with OPERAND = 0x05. For 0x000 all enables stay low (NOP).

4.7 Control unit (control_unit)

The control unit is a clocked block that generates registered enable signals from instruction[11:8] and the PC control signals PC_RESET and PC_LOAD (PC_LOAD is set for JZ). On RESET all enables are cleared and PC_RESET is set. In cpu_top the datapath uses the decoder's combinational enables so each instruction finishes in one clock cycle, while this unit provides the registered control and status signals.

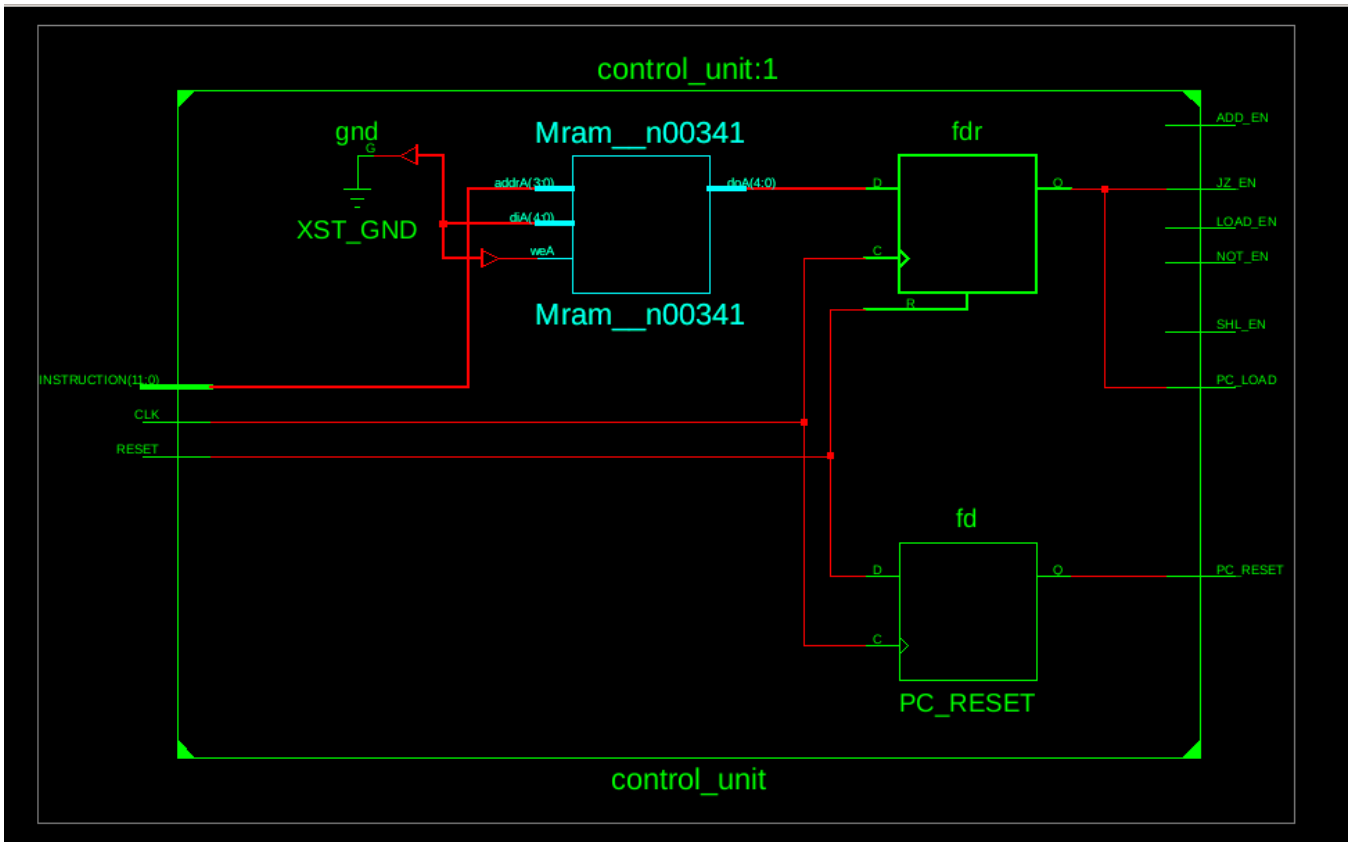


Figure 14: *Block symbol of control_unit*

VHDL code: control_unit.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity control_unit is
    Port (
        CLK         : in  STD_LOGIC;
        RESET       : in  STD_LOGIC;
        INSTRUCTION : in  STD_LOGIC_VECTOR(11 downto 0);

        LOAD_EN     : out STD_LOGIC;
        ADD_EN      : out STD_LOGIC;
        NOT_EN      : out STD_LOGIC;
        SHL_EN      : out STD_LOGIC;
        JZ_EN       : out STD_LOGIC;
        PC_RESET    : out STD_LOGIC;
        PC_LOAD     : out STD_LOGIC
    );
end control_unit;
architecture Behavioral of control_unit is
begin

    process(CLK)
    begin
        if rising_edge(CLK) then
            if RESET = '1' then
                LOAD_EN  <= '0';
                ADD_EN   <= '0';
                NOT_EN   <= '0';
                SHL_EN   <= '0';
                JZ_EN    <= '0';
                PC_RESET <= '1';
                PC_LOAD  <= '0';
            else
                LOAD_EN  <= '0';
                ADD_EN   <= '0';
                NOT_EN   <= '0';
                SHL_EN   <= '0';
                JZ_EN    <= '0';
                PC_RESET <= '0';
                PC_LOAD  <= '0';

                case INSTRUCTION(11 downto 8) is

                    when "0001" =>
                        LOAD_EN <= '1';

                    when "0010" =>
                        ADD_EN <= '1';
                    when "0111" =>
```

```
                        NOT_EN <= '1';

                    when "1000" =>
                        SHL_EN <= '1';

                    when "1100" =>
                        JZ_EN   <= '1';
                        PC_LOAD <= '1';

                    when others =>
                        null;

                end case;
            end if;
        end if;
    end process;

end Behavioral;
```

Testbench stimulus: control_unit_tb.vhd (excerpt)

```
    STIMULUS: process
    begin
        -- Test RESET
        RESET <= '1';
        INSTRUCTION <= x"000";
        wait until rising_edge(CLK);
        wait for 1 ns;

        assert PC_RESET = '1'
            report "RESET test failed"
            severity error;

        RESET <= '0';
        -- Test LOAD instruction
        INSTRUCTION <= x"1FF";
        wait until rising_edge(CLK);
        wait for 1 ns;

        assert LOAD_EN = '1' and ADD_EN = '0'
            report "LOAD instruction test failed"
            severity error;

        -- Test ADD instruction
        INSTRUCTION <= x"277";
        wait until rising_edge(CLK);
        wait for 1 ns;
        assert ADD_EN = '1' and LOAD_EN = '0'
            report "ADD instruction test failed"
            severity error;

        -- Test NOT instruction
        INSTRUCTION <= x"700";
        wait until rising_edge(CLK);
        wait for 1 ns;

        assert NOT_EN = '1'
            report "NOT instruction test failed"
            severity error;
        -- Test SHL instruction
        INSTRUCTION <= x"800";
        wait until rising_edge(CLK);
        wait for 1 ns;

        assert SHL_EN = '1'
            report "SHL instruction test failed"
            severity error;

        -- Test JZ instruction
        INSTRUCTION <= x"C05";
        wait until rising_edge(CLK);
        wait for 1 ns;
        assert JZ_EN = '1' and PC_LOAD = '1'
            report "JZ instruction test failed"
            severity error;

        -- Test NOP instruction
        INSTRUCTION <= x"000";
        wait until rising_edge(CLK);
        wait for 1 ns;
        assert LOAD_EN = '0' and
               ADD_EN  = '0' and
               NOT_EN  = '0' and
               SHL_EN  = '0' and
               JZ_EN   = '0' and
               PC_LOAD = '0'
            report "Invalid/NOP instruction test failed"
            severity error;

        report "Control unit test completed successfully."
            severity note;

        wait;
    end process;
```

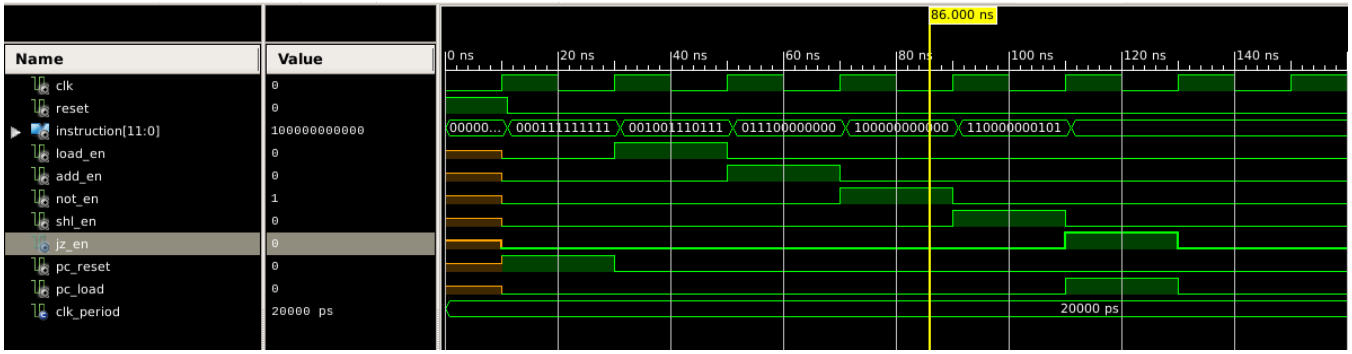


Figure 15: *Simulation waveform of control_unit*

While RESET = 1, PC_RESET goes high at the rising edge. Afterwards each instruction (0x1FF, 0x277, 0x700, 0x800, 0xC05) produces its own enable pulse (LOAD_EN, ADD_EN, NOT_EN, SHL_EN, JZ_EN) at the next rising edge, with PC_LOAD asserted together with JZ_EN. The NOP (0x000) produces no enable.

4.8 Top-level CPU (cpu_top)

cpu_top connects all blocks as shown in Figure 1. Two small processes implement the selection logic: the ACC next-value process chooses the value to be stored (the operand for LDI, the ALU result for ADD, the inverted ACC for NOT, ACC shifted left with a 0 in the LSB for SHL, otherwise ACC unchanged), and the next-PC process chooses between 0 (RESET), the JZ target (when JZ_EN = 1 and Z = 1) and PC + 1. The accumulator write enable is the OR of the LOAD, ADD, NOT and SHL enables, so JZ and NOP leave ACC unchanged. The outputs are Q = ACC, Z and PC.

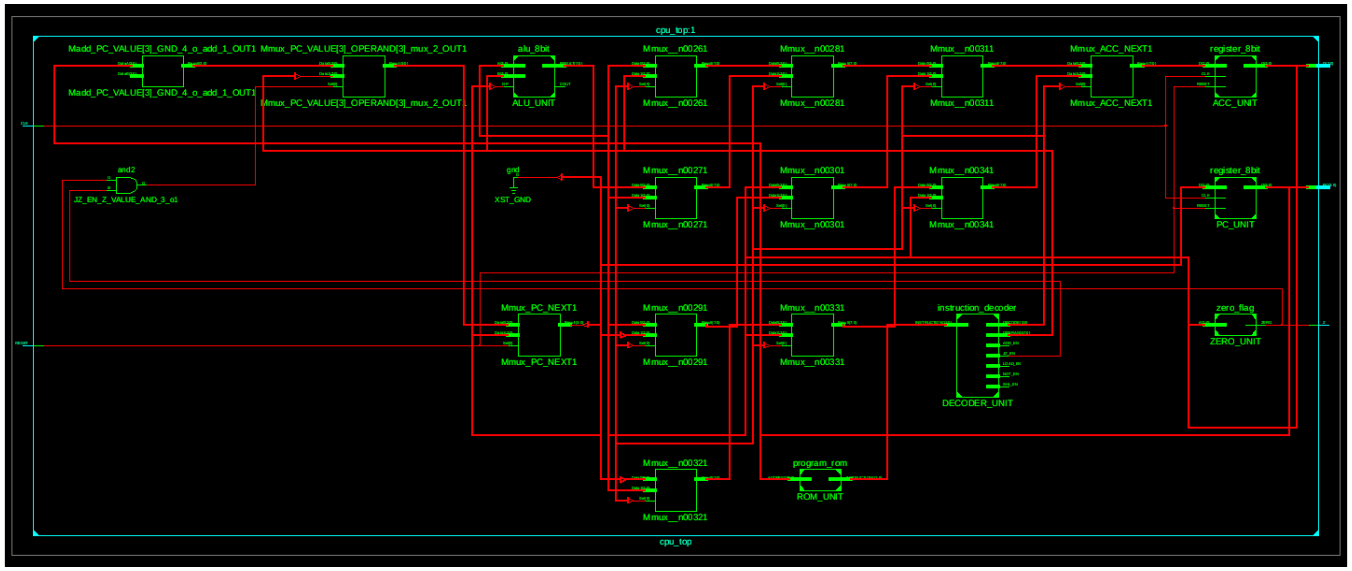


Figure 16: *Block symbol of cpu_top*

VHDL code: cpu_top.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity cpu_top is
    Port (
        CLK   : in  STD_LOGIC;
        RESET : in  STD_LOGIC;
        Q     : out STD_LOGIC_VECTOR(7 downto 0);
        Z     : out STD_LOGIC;
        PC    : out STD_LOGIC_VECTOR(3 downto 0)
    );
end cpu_top;
architecture Structural of cpu_top is

    signal PC_VALUE      : STD_LOGIC_VECTOR(3 downto 0);
    signal PC_NEXT       : STD_LOGIC_VECTOR(3 downto 0);
       signal PC_REG_Q      : STD_LOGIC_VECTOR(7 downto 0);

       signal ACC_NEXT      : STD_LOGIC_VECTOR(7 downto 0);
    signal ACC_VALUE     : STD_LOGIC_VECTOR(7 downto 0);
```

```
    signal ALU_VALUE               : STD_LOGIC_VECTOR(7 downto 0);
    signal INSTRUCTION   : STD_LOGIC_VECTOR(11 downto 0);
    signal OPCODE        : STD_LOGIC_VECTOR(3 downto 0);
    signal OPERAND       : STD_LOGIC_VECTOR(7 downto 0);

    signal LOAD_EN       : STD_LOGIC;
    signal ADD_EN        : STD_LOGIC;
    signal NOT_EN        : STD_LOGIC;
    signal SHL_EN        : STD_LOGIC;
    signal JZ_EN         : STD_LOGIC;
    signal PC_RESET      : STD_LOGIC;
    signal PC_LOAD       : STD_LOGIC;

    signal ACC_WRITE_EN  : STD_LOGIC;
    signal Z_VALUE       : STD_LOGIC;
    signal Z_NEXT        : STD_LOGIC;

    -- ALU signals
    signal ALU_RESULT    : STD_LOGIC_VECTOR(7 downto 0);
    signal ALU_COUT      : STD_LOGIC;

begin
    -------------------------------------------------
    -- 1. Program ROM
    -------------------------------------------------
    ROM_UNIT : entity work.program_rom
        port map (
            ADDRESS     => PC_VALUE,
            INSTRUCTION => INSTRUCTION
        );
    -------------------------------------------------
    -- 2. Instruction Decoder
    -------------------------------------------------
    DECODER_UNIT : entity work.instruction_decoder
        port map (
            INSTRUCTION => INSTRUCTION,
            OPCODE      => OPCODE,
            OPERAND     => OPERAND,
            LOAD_EN     => LOAD_EN,
            ADD_EN      => ADD_EN,
            NOT_EN      => NOT_EN,
            SHL_EN      => SHL_EN,
            JZ_EN       => JZ_EN
        );
    -------------------------------------------------
    -- 3. Control Unit
    -------------------------------------------------
    CONTROL_UNIT : entity work.control_unit
        port map (
            CLK         => CLK,
            RESET       => RESET,
            INSTRUCTION => INSTRUCTION,
            LOAD_EN     => open,
            ADD_EN      => open,
            NOT_EN      => open,
            SHL_EN      => open,
            JZ_EN       => open,
            PC_RESET    => PC_RESET,
            PC_LOAD     => PC_LOAD
        );
    -------------------------------------------------
    -- 4. ALU
    -------------------------------------------------
    ALU_UNIT : entity work.alu_8bit
        port map (
            A      => ACC_VALUE,
            B      => OPERAND,
            OP     => '0',
            RESULT => ALU_RESULT,
            COUT   => ALU_COUT
        );
    -------------------------------------------------
    -- 5. Accumulator next-value logic
    -------------------------------------------------
    process(ACC_VALUE, OPERAND, OPCODE, ALU_RESULT)
    begin
        -- Default: preserve current accumulator
        ACC_NEXT <= ACC_VALUE;

        case OPCODE is
            when "0001" =>
                ACC_NEXT <= OPERAND;     -- LDI

            when "0010" =>
                ACC_NEXT <= ALU_RESULT;  -- ADD
            when "0111" =>
                ACC_NEXT <= ALU_VALUE;  -- NOT

            when "1000" =>
                ACC_NEXT <= ACC_VALUE(6 downto 0) & '0';  -- SHL

            when others =>
                ACC_NEXT <= ACC_VALUE;   -- JZ or NOP
        end case;
    end process;

    ACC_WRITE_EN <= LOAD_EN or ADD_EN or NOT_EN or SHL_EN;
    -------------------------------------------------
```

```
    -- 6. Accumulator
    ------------------------------------------------
    ACC_UNIT : entity work.accumulator
        port map (
            D     => ACC_NEXT,
            CLK   => CLK,
            RESET => RESET,
            Q     => ACC_VALUE
        );
    ------------------------------------------------
    -- 7. Zero Flag
    ------------------------------------------------
    ZERO_UNIT : entity work.zero_flag
        port map (
            A    => ACC_VALUE,
            ZERO => Z_VALUE
        );
    ------------------------------------------------
    -- 8. Program Counter next-value logic
    ------------------------------------------------
    process(PC_VALUE, OPERAND, JZ_EN, Z_VALUE, RESET)
    begin
        -- Normal instruction: move to next address
        PC_NEXT <= std_logic_vector(unsigned(PC_VALUE) + 1);
        if RESET = '1' then
            PC_NEXT <= (others => '0');

        elsif JZ_EN = '1' and Z_VALUE = '1' then
            -- JZ: jump only when zero flag is set
            PC_NEXT <= OPERAND(3 downto 0);

        end if;
    end process;
    ------------------------------------------------
    -- 9. Program Counter
    ------------------------------------------------
    PC_UNIT : entity work.program_counter
                port map (
                         CLK   => CLK,
                         RESET => RESET,
                         LOAD  => JZ_EN and Z_VALUE,
                         D     => OPERAND(3 downto 0),
                         PC    => PC_VALUE
                );
    ------------------------------------------------
    -- 10. Outputs
    ------------------------------------------------
    Q  <= ACC_VALUE;
    Z  <= Z_VALUE;
    PC <= PC_VALUE;

end Structural;
```

5. Top-Level Testbench and Results

The testbench generates a 20 ns clock, applies RESET for one rising edge, and then runs one clock cycle for each instruction of the Set 9 program. One nanosecond after every edge it checks Q (ACC), Z and PC with assert statements against the expected Appendix B values. The complete testbench code follows.

Testbench code: cpu_top_tb.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY cpu_top_tb IS
END cpu_top_tb;

ARCHITECTURE behavior OF cpu_top_tb IS

    COMPONENT cpu_top
        PORT (
            CLK   : IN  std_logic;
            RESET : IN  std_logic;
            Q     : OUT std_logic_vector(7 downto 0);
            Z     : OUT std_logic;
            PC    : OUT std_logic_vector(3 downto 0)
        );
    END COMPONENT;
    -- Inputs
    signal CLK   : std_logic := '0';
    signal RESET : std_logic := '0';

    -- Outputs
    signal Q  : std_logic_vector(7 downto 0);
    signal Z  : std_logic;
    signal PC : std_logic_vector(3 downto 0);

    constant CLK_PERIOD : time := 20 ns;

BEGIN
    -- Instantiate the CPU
    uut : cpu_top
```

```
        PORT MAP (
            CLK   => CLK,
            RESET => RESET,
            Q     => Q,
            Z     => Z,
            PC    => PC
        );

    -- Clock generation: 20 ns period
    clk_process : process
    begin
        CLK <= '0';
        wait for CLK_PERIOD / 2;
        CLK <= '1';
        wait for CLK_PERIOD / 2;
    end process;
    -- Set 9 test sequence
    stim_proc : process
    begin
        -- Apply synchronous reset
        RESET <= '1';
        wait for CLK_PERIOD;
        wait for 1 ns;

        assert Q = x"00" and Z = '1' and PC = "0000"
            report "FAIL: Reset state"
            severity error;

        RESET <= '0';
        -- Instruction 0: LDI FF
        wait for CLK_PERIOD;
        wait for 1 ns;

        assert Q = x"FF" and Z = '0' and PC = "0001"
            report "FAIL: LDI FF"
            severity error;

        -- Instruction 1: NOT
        wait for CLK_PERIOD;
        wait for 1 ns;
        assert Q = x"00" and Z = '1' and PC = "0010"
            report "FAIL: NOT"
            severity error;

        -- Instruction 2: JZ 05 (taken)
        wait for CLK_PERIOD;
        wait for 1 ns;

        assert Q = x"00" and Z = '1' and PC = "0101"
            report "FAIL: JZ 05"
            severity error;
        -- Instruction 5: ADD 11
        wait for CLK_PERIOD;
        wait for 1 ns;

        assert Q = x"11" and Z = '0' and PC = "0110"
            report "FAIL: ADD 11"
            severity error;

        -- Instruction 6: SHL
        wait for CLK_PERIOD;
        wait for 1 ns;
        assert Q = x"22" and Z = '0' and PC = "0111"
            report "FAIL: SHL at address 6"
            severity error;

        -- Instruction 7: JZ 00 (not taken)
        wait for CLK_PERIOD;
        wait for 1 ns;

        assert Q = x"22" and Z = '0' and PC = "1000"
            report "FAIL: JZ 00 not taken"
            severity error;
        -- Instruction 8: SHL
        wait for CLK_PERIOD;
        wait for 1 ns;

        assert Q = x"44" and Z = '0' and PC = "1001"
            report "FAIL: SHL at address 8"
            severity error;

        report "Set 9 CPU testbench completed."
            severity note;

        wait;
    end process;

END behavior;
```

5.1 Simulation waveform

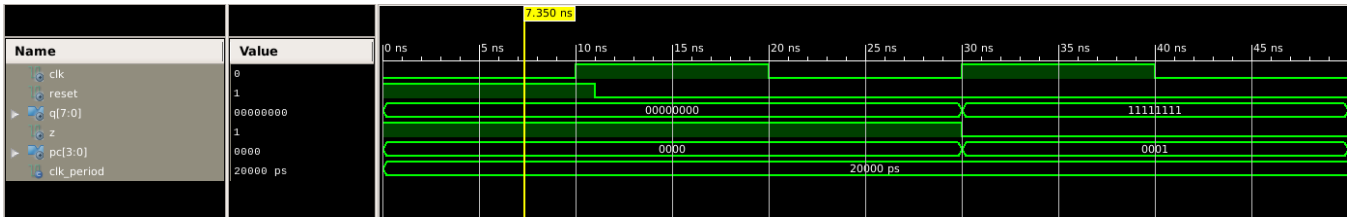


Figure 17: *Waveform of cpu_top running the Set 9 program (CLK, RESET, PC, current instruction, Q = ACC, Z)*

RESET is high at the first rising edge (10 ns), so PC = 0x0, ACC = 0x00 and Z = 1. The program then runs one instruction per rising edge:

- **LDI 0xFF** loads ACC = 0xFF and Z becomes 0.
- **NOT** inverts ACC to 0x00 and Z becomes 1.
- **JZ 0x05** is taken because Z = 1: PC jumps from 2 to 5 and ACC is unchanged. Addresses 3 and 4 are skipped.
- **ADD 0x11** gives ACC = 0x11 and Z = 0.
- **SHL** gives ACC = 0x22.
- **JZ 0x00** is not taken because Z = 0: PC simply goes from 7 to 8, ACC stays 0x22.
- **SHL** gives ACC = 0x44 and PC = 9.

5.2 Result check against Appendix B

Table 4: *Simulated values after each rising edge compared with the Appendix B table for Set 9*

Cycle	PC	Instruction executed	ACC (hex)	ACC (binary)	Z	Next PC	Status
0	-	RESET = 1	0x00	00000000	1	0	Pass
1	0	LDI 0xFF	0xFF	11111111	0	1	Pass
2	1	NOT	0x00	00000000	1	2	Pass
3	2	JZ 0x05 (taken)	0x00	00000000	1	5	Pass
4	5	ADD 0x11	0x11	00010001	0	6	Pass
5	6	SHL	0x22	00100010	0	7	Pass
6	7	JZ 0x00 (not taken)	0x22	00100010	0	8	Pass
7	8	SHL	0x44	01000100	0	9	Pass

All eight cycles match the expected ACC, Z and next-PC values of Appendix B (Set 9). The testbench finished with the message "Set 9 CPU testbench completed" and no assertion failure.

6. Challenges Faced

- **Building everything from NAND gates.** The gate-level hierarchy is deep (NAND, gates, adders, ALU, CPU). Debugging was done bottom-up: every module got its own testbench and was verified before it was used in a larger block.
- **Interface mismatch between ALU and opcode.** The ALU select input OP is one bit (add / subtract) while the decoder produces a 4-bit opcode, which caused a type error in the synthesis report. The ALU is only needed as an adder for ADD, so OP is tied to 0 in cpu_top.
- **Program counter had no load input.** A free-running incrementing PC cannot implement JZ. The PC was changed to a register with data input D, and the PC + 1 / jump-target selection was moved to the next-PC logic in cpu_top.
- **Accumulator behaviour.** The first accumulator added its input to its own value, which is wrong for LDI, NOT and SHL. It was redesigned as a register with a hold/load multiplexer and write enable, while the ACC next-value logic selects the new value for each opcode.

- **NOT and SHL are not adder operations.** These two results are generated in the ACC next-value process (bitwise inverse and a one-bit left shift) instead of in the adder.
- **Timing of Z and the checks.** Z is derived directly from ACC, so it is 1 after reset and JZ uses the flag produced by the previous instruction. The testbench samples outputs 1 ns after the rising edge to avoid race conditions.
- **Control path.** Using only registered enables would delay each write by one cycle. The combinational decoder enables drive the datapath so that every instruction completes in a single clock cycle.

7. Conclusion

A single-cycle 8-bit accumulator CPU for Set 9 (LDI, ADD, **NOT**, **SHL**, **JZ**) was designed in VHDL using a modular, gate-level hierarchy built from NAND gates. The design contains separate entities for the PC, Program ROM, decoder, control unit, ALU, accumulator and zero flag, connected in a top-level cpu_top entity with synchronous active-high reset and outputs CLK, RESET, Q[7:0], Z and PC[3:0]. The Appendix B program was stored in the ROM and executed by the testbench: ACC, Z and PC match the expected values on every cycle, including the taken and the not-taken JZ. The project showed how a complete processor can be built step by step from one primitive gate and verified at each level.

8. Submitted Files

- **VHDL entities:** nand_gate, not_gate, and_gate, or_gate, xor_gate, half_adder, full_adder, full_adder_8bit, register_8bit, mux_8bit, alu_8bit, zero_flag, accumulator, program_counter, program_rom, instruction_decoder, control_unit, cpu_top (.vhd).
- **Testbenches:** one _tb.vhd file for every entity, including cpu_top_tb.vhd for the Set 9 program.

9. Github Link

https://github.com/Tasnim3322Abrar/ISE-14.7-for-VLSI/tree/main/cpu_design